

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Báo Cáo Đồ Án 1

Đề tài: Xây dựng CI Pipeline cho hệ thống YAS — Yet Another Shop

Môn học: Nhập môn DevOps

Sinh viên thực hiện:

Ngô Gia An (23120205)

Nguyễn Bảo An (23120207)

Dương Tuấn Anh (23120208)

Trương Nhật Đạt (23120231)

Ngày 2 tháng 5 năm 2026

Mục lục

1	Thông Tin Nhóm	1
2	Checklist Yêu Cầu Đồ Án	1
2.1	Yêu cầu bắt buộc	1
2.2	Yêu cầu nâng cao	1
2.3	Yêu cầu nộp bài	2
3	Phân Công Công Việc	2
4	Tổng Quan Kiến Trúc Pipeline	2
5	Chi Tiết Triển Khai	3
5.1	Jenkins Infrastructure & Pipeline	3
5.1.1	Cài Đặt Jenkins Server	3
5.1.2	Cấu Hình GitHub Webhook	4
5.1.3	Thiết Lập Multibranch Pipeline	6
5.1.4	Nội Dung Jenkinsfile	9
5.1.5	Kiểm Tra Pipeline Hoạt Động	11
5.1.6	Cấu Hình Agent Pool (Distributed Builds)	12
5.2	Branch Protection	13
5.2.1	Cấu Hình Branch Protection Trên GitHub	13
5.2.2	Pull Request Demo (Trạng Thái Open)	16
5.3	Security Scanning	17
5.3.1	Tích Hợp Gitleaks — Quét Secret Bị Lộ	17
5.3.2	Tích Hợp SonarQube — Phân Tích Chất Lượng Code	20
5.3.3	Tích Hợp Snyk — Quét Lỗ Hổng Dependency	23
5.4	Coverage Gate	25
5.4.1	Cấu Hình Trong pom.xml	25
5.4.2	Cấu Hình Stage Trong Jenkinsfile	26
5.4.3	Hình Ảnh Minh Chứng	27

6	Unit Test — Chi Tiết Từng Module	29
6.1	Hướng Dẫn Chung Chạy Test	29
6.2	Module <code>media</code>	29
6.3	Module <code>product</code>	31
6.4	Module <code>order</code>	31
6.5	Module <code>inventory</code>	32
6.6	Module <code>payment</code>	33
6.7	Module <code>promotion</code>	33
6.8	Module <code>rating</code>	34
6.9	Module <code>delivery</code>	35
6.10	Module <code>sampledata</code>	35
6.11	Module <code>recommendation</code>	36
6.12	Module <code>customer</code>	36
6.13	Module <code>location</code>	37
6.14	Module <code>cart</code>	38
6.15	Module <code>tax</code>	38
6.16	Module <code>search</code>	39
6.17	Module <code>webhook</code>	39
6.18	Module <code>common-library</code>	40
7	Tổng Hợp Kết Quả Coverage (Toàn Dự Án — 17 Modules)	41

Danh sách hình vẽ

1	Jenkins Dashboard sau khi cài đặt thành công	4
2	Cấu hình Webhook trên GitHub	5
3	Cấu hình trigger event Pull Request cho Webhook	5
4	Webhook delivery thành công (ping event trả về HTTP 200)	6
5	Cấu hình Multibranch Pipeline Job (1)	7
6	Cấu hình Multibranch Pipeline Job (2)	7
7	Cấu hình Personal Access Token cho Multibranch Pipeline	8
8	Multibranch Pipeline — danh sách branch được Jenkins phát hiện	8
9	Pipeline tự động kích hoạt sau khi push code	9
10	Tất cả stage pipeline chạy thành công	11
11	Lịch sử build trên Jenkins	12
12	Danh sách Nodes trên Jenkins: 4 agent + Built-In Node	13
13	Cấu hình bắt buộc tạo Pull Request và số lượng Approval	14
14	Cấu hình bắt buộc Status Checks (Jenkins CI) phải pass	15
15	Cấu hình bắt buộc cập nhật nhánh trước khi merge	15
16	Cấu hình không cho phép Admin lách luật	15
17	Push trực tiếp vào nhánh main bị từ chối	16
18	Pull Request hiển thị yêu cầu 2 lượt approve và CI check phải pass	16
19	Pull Request #33 đang ở trạng thái Open, yêu cầu 2 approvals và CI pass	17
20	Stage Secret Scanning chạy thành công (không phát hiện secret)	19
21	Pipeline thất bại khi Gitleaks phát hiện secret	19
22	SonarQube Dashboard: project yas với Quality Gate Passed	21
23	Jenkins pipeline: stage Code Quality và Quality Gate chạy thành công	22
24	Jenkins build summary: pipeline hoàn thành thành công	22
25	Snyk Dashboard: danh sách project và vulnerability được phát hiện	24
26	Stage Dependency Scan trong Jenkins pipeline chạy thành công	24
27	Toàn bộ pipeline chạy thành công với đầy đủ Security Stages	25
28	Cấu hình JaCoCo Coverage Gate trong file pom.xml	27
29	Pipeline thất bại khi coverage dưới ngưỡng 70%	28

30	Pipeline thành công sau khi bổ sung đủ unit test	28
31	Báo cáo JaCoCo Coverage hiển thị trong giao diện Jenkins	29
32	Báo cáo JaCoCo Coverage — module media (80%)	30
33	Báo cáo JaCoCo Coverage — module product (71%)	31
34	Báo cáo JaCoCo Coverage — module order (76%)	32
35	Báo cáo JaCoCo Coverage — module inventory (89%)	33
36	Báo cáo JaCoCo Coverage — module payment (72.33%)	33
37	Báo cáo JaCoCo Coverage — module promotion (82.07%)	34
38	Báo cáo JaCoCo Coverage — module rating (84.56%)	35
39	Báo cáo JaCoCo Coverage — module delivery (100%)	35
40	Báo cáo JaCoCo Coverage — module sampledata (81.37%)	36
41	Báo cáo JaCoCo Coverage — module recommendation (86.32%)	36
42	Báo cáo JaCoCo Coverage — module customer (87%)	37
43	Báo cáo JaCoCo Coverage — module location (87%)	38
44	Báo cáo JaCoCo Coverage — module cart (88%)	38
45	Báo cáo JaCoCo Coverage — module tax (87%)	39
46	Báo cáo JaCoCo Coverage — module search (85%)	39
47	Báo cáo JaCoCo Coverage — module webhook (78%)	40
48	Báo cáo JaCoCo Coverage — module common-library (93.9%)	40

1 Thông Tin Nhóm

Bảng 1: Thông tin thành viên nhóm

Thành viên	MSSV	Phân phụ trách
Ngô Gia An	23120205	Jenkins Infrastructure và Pipeline
Nguyễn Bảo An	23120207	Branch Protection và Unit Test (10 modules)
Đương Tuấn Anh	23120208	Security Scanning
Trương Nhật Đạt	23120231	Coverage Gate và Unit Test (6 modules)

Link GitHub Repository: <https://github.com/com-suon-bi-cha/yas>

Link Pull Request (Open): <https://github.com/com-suon-bi-cha/yas/pull/33>

2 Checklist Yêu Cầu Đồ Án

2.1 Yêu cầu bắt buộc

Bảng 2: Checklist yêu cầu bắt buộc

#	Yêu cầu	Trạng thái
1	Công cụ CI/CD: Jenkins	Hoàn thành
2	Fork repository về GitHub của nhóm	Hoàn thành
3	Branch Protection: cấm push thẳng vào <code>main</code> , yêu cầu 2 approvals + CI pass	Hoàn thành
4	Multibranch Pipeline: Jenkins tự quét và chạy pipeline cho từng branch	Hoàn thành
5	Pipeline có ít nhất 2 giai đoạn: Test và Build	Hoàn thành
6	Upload kết quả test và báo cáo độ phủ	Hoàn thành
7	Monorepo Optimization: chỉ build/test service có thay đổi	Hoàn thành

2.2 Yêu cầu nâng cao

Bảng 3: Checklist yêu cầu nâng cao

#	Yêu cầu	Trạng thái
a	Unit test cho từng service, tạo branch riêng biệt	Hoàn thành
b	Pipeline fail khi coverage < 70% (JaCoCo Gate)	Hoàn thành
c	Tích hợp Gitleaks, SonarQube, Snyk	Hoàn thành

2.3 Yêu cầu nộp bài

Bảng 4: Thông tin nộp bài

#	Yêu cầu	Giá trị
1	Link GitHub Repository	https://github.com/com-suon-bi-cha/yas
2	Pull Request đang ở trạng thái Open	https://github.com/com-suon-bi-cha/yas/pull/33
3	File báo cáo	23120205_23120207_23120208_23120231.docx

3 Phân Công Công Việc

Bảng 5: Phân công công việc các thành viên

Thành viên	Nội dung thực hiện
Ngô Gia An	Cài đặt Jenkins Server, cấu hình Webhook, tạo Multibranch Pipeline, viết Jenkinsfile với logic Monorepo
Nguyễn Bảo An	Cấu hình Branch Protection trên GitHub, viết unit test cho 10 service module (media, product, order, inventory, payment, promotion, rating, delivery, sampledata, recommendation)
Dương Tuấn Anh	Tích hợp Gitleaks, SonarQube, Snyk vào pipeline
Trương Nhật Đạt	Cấu hình JaCoCo Coverage Gate ($\geq 70\%$), viết unit test cho 6 service module (customer, location, cart, tax, search, webhook), tổng hợp báo cáo

4 Tổng Quan Kiến Trúc Pipeline

Nhóm triển khai CI pipeline theo luồng sau:

```

1 Developer Push / Pull Request
2   |
3   v
4   GitHub Webhook
5   |
6   v
7   Jenkins (Multibranch Pipeline)
8   |
9   |-- Stage 1: Pre-check          (Kiểm tra Java, Maven, Gitleaks,
    Snyk)
    
```

```
10      |-- Stage 2: Secret Scanning      (Gitleaks)
11      |-- Stage 3: Monorepo Execution (Test + Build chỉ service có thay
    doi)
12      |-- Stage 4: Code Quality          (SonarQube)
13      |-- Stage 5: Quality Gate          (SonarQube Quality Gate)
14      |-- Stage 6: Coverage Report       (JaCoCo >= 70%)
15      |-- Stage 7: Dependency Scan       (Snyk)
16      |
17      v
18  GitHub Branch Protection
19  (Yêu cầu 2 approvals + CI Pass mới cho phép merge)
```

Listing 1: Luồng CI Pipeline

Tối ưu hóa Monorepo: Pipeline chỉ kích hoạt build/test cho service có thay đổi, sử dụng git diff để phát hiện thư mục bị ảnh hưởng.

5 Chi Tiết Triển Khai

5.1 Jenkins Infrastructure & Pipeline

5.1.1 Cài Đặt Jenkins Server

Bảng 6: Thông số môi trường Jenkins

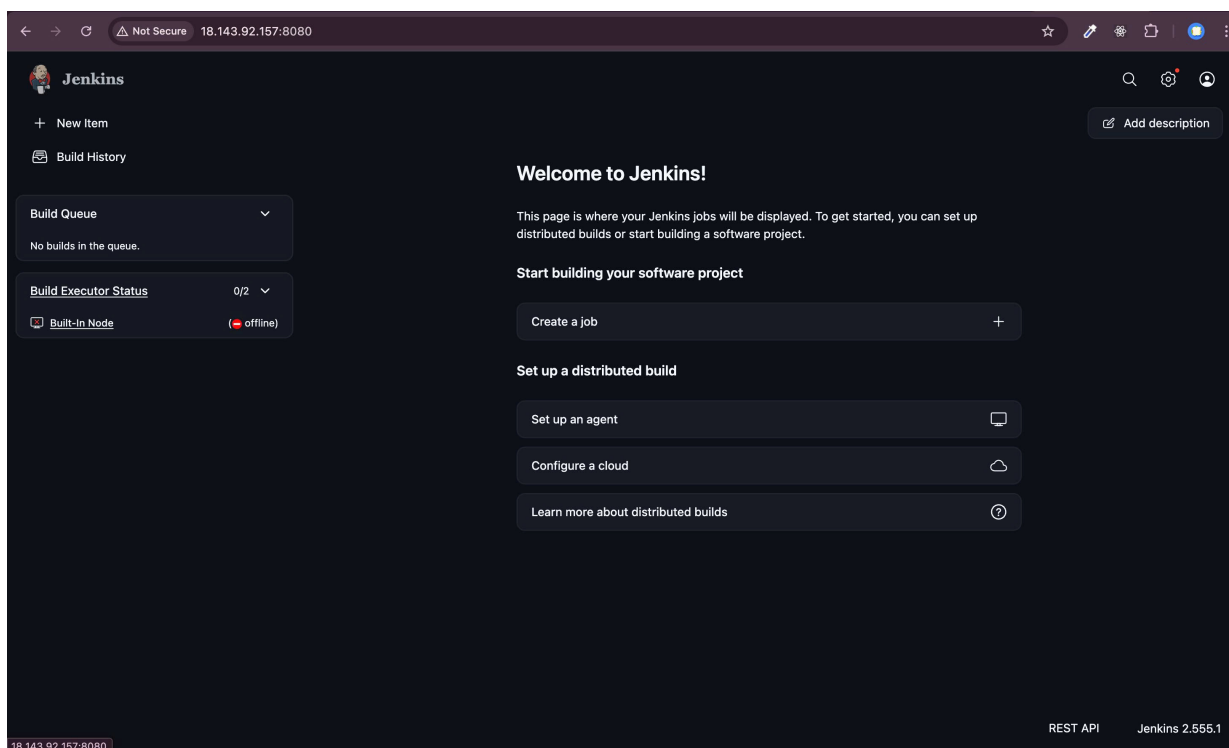
Thông số	Giá trị
Phương thức triển khai	Virtual Machine Amazon EC2
Phiên bản Jenkins	2.555.1
Hệ điều hành	Amazon Linux 2023 (kernel 6.1 AMI, architecture: arm64, instance: t3.small)
Phiên bản Java	Amazon Corretto 21 (OpenJDK 21.0.10 LTS)

Môi Trường Triển Khai

Bảng 7: Các plugin Jenkins đã cài đặt

Plugin	Mục đích
Git Plugin	Kết nối với GitHub repository
Pipeline	Chạy Jenkinsfile dạng Declarative/Scripted
Multibranch Pipeline	Tự động quét và build nhiều branch
JaCoCo Plugin	Publish báo cáo độ phủ unit test
Warnings Next Generation	Hiển thị kết quả phân tích tĩnh
GitHub Integration Plugin	Nhận sự kiện từ GitHub Webhook

Plugin Đã Cài Đặt Ngoài ra còn một số plugin được Jenkins recommend khi setup lần đầu tiên.



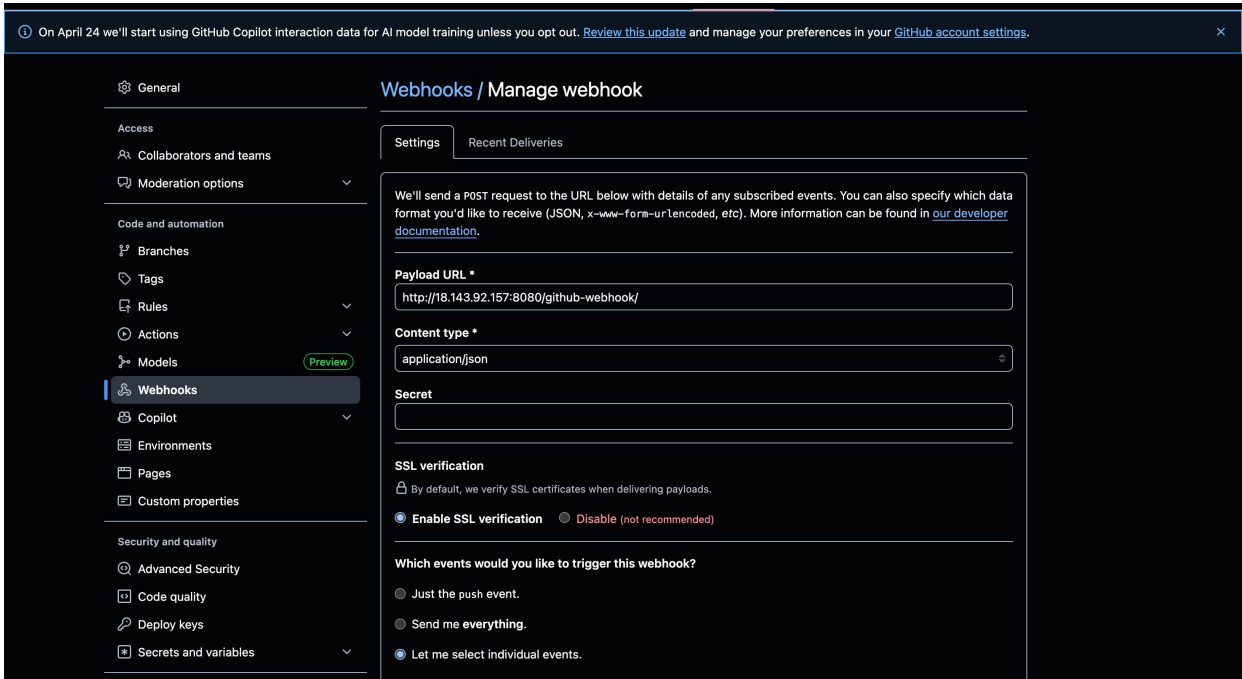
Hình 1: Jenkins Dashboard sau khi cài đặt thành công

5.1.2 Cấu Hình GitHub Webhook

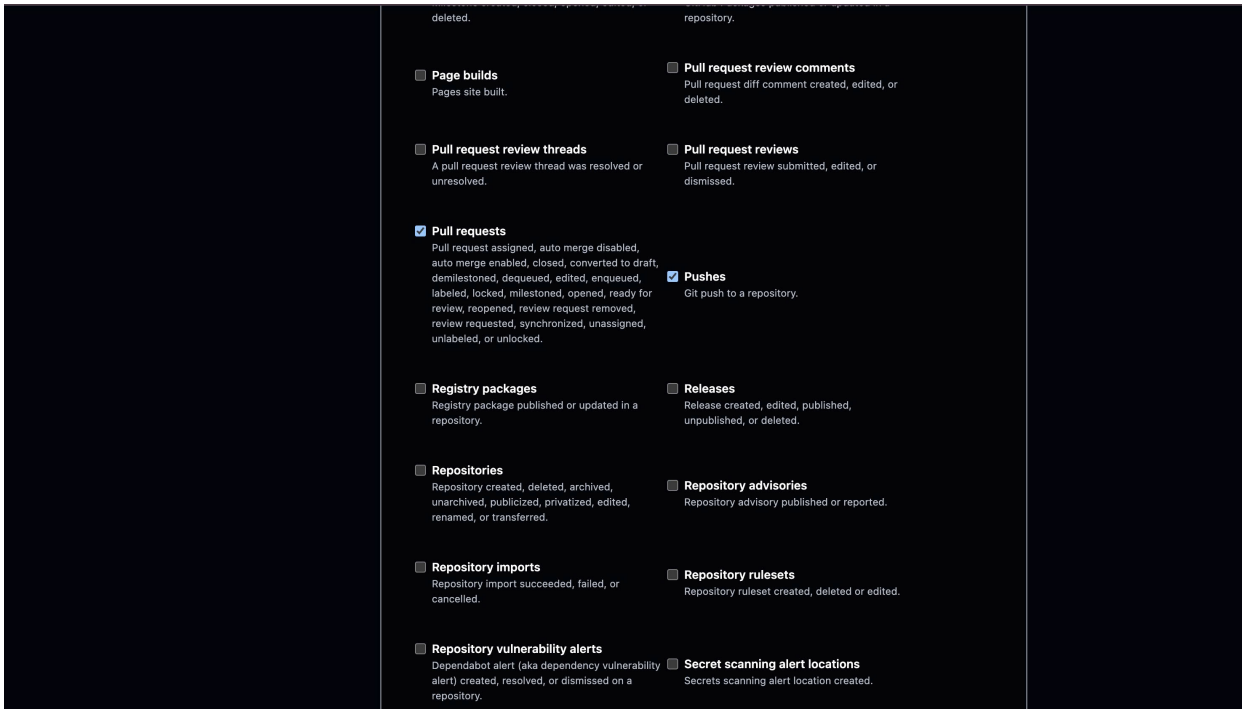
Webhook được tạo tại: Repository > Settings > Webhooks > Add webhook

Bảng 8: Cấu hình GitHub Webhook

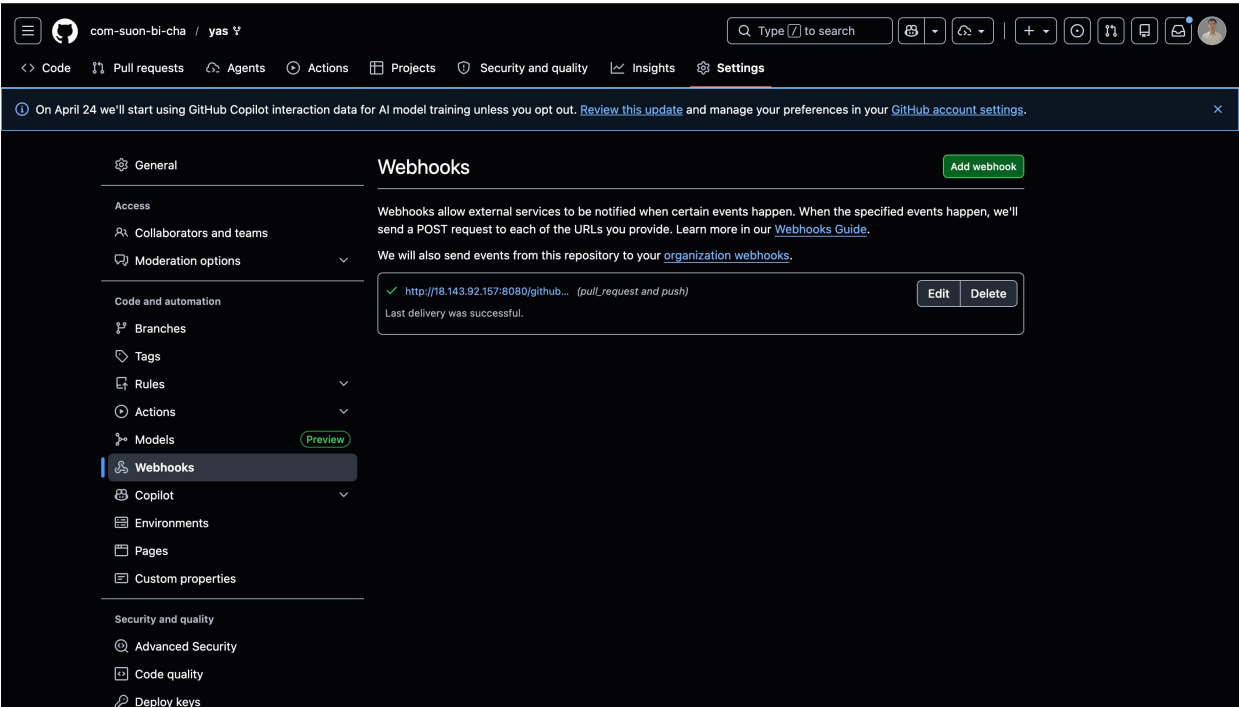
Cấu hình	Giá trị
Payload URL	http://18.143.92.157:8080/github-webhook/
Content type	application/json
Trigger events	Push events, Pull request events
Trạng thái	Active



Hình 2: Cấu hình Webhook trên GitHub



Hình 3: Cấu hình trigger event Pull Request cho Webhook

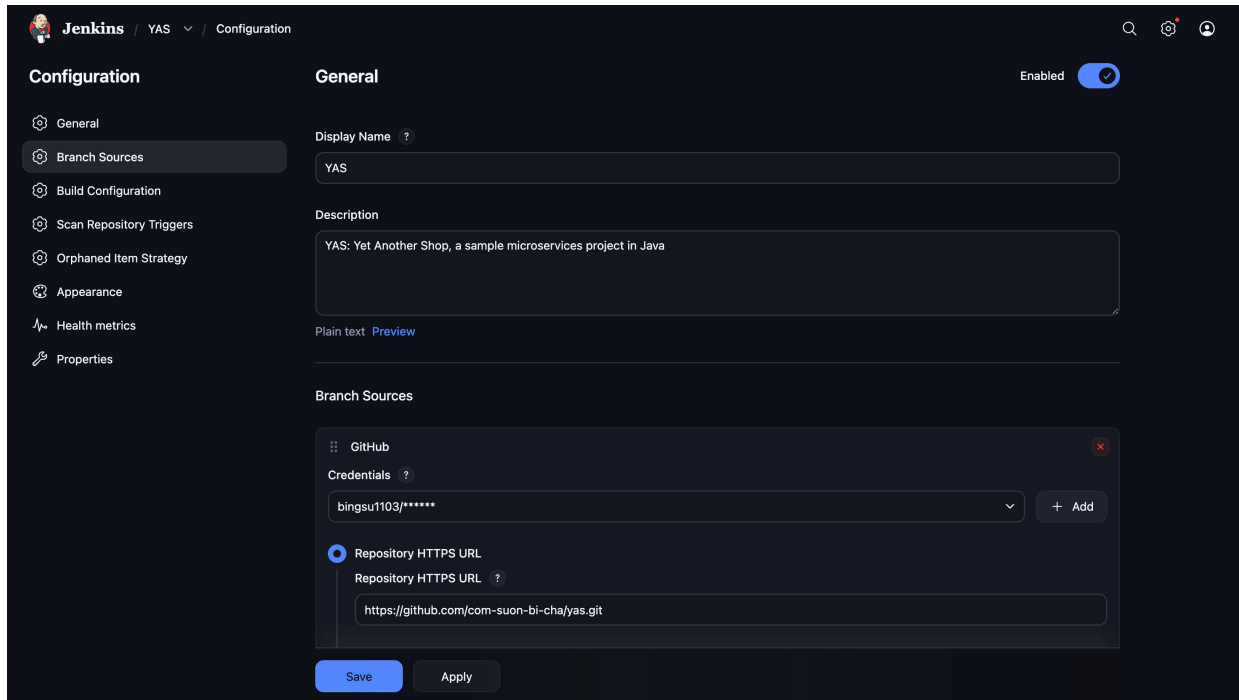


Hình 4: Webhook delivery thành công (ping event trả về HTTP 200)

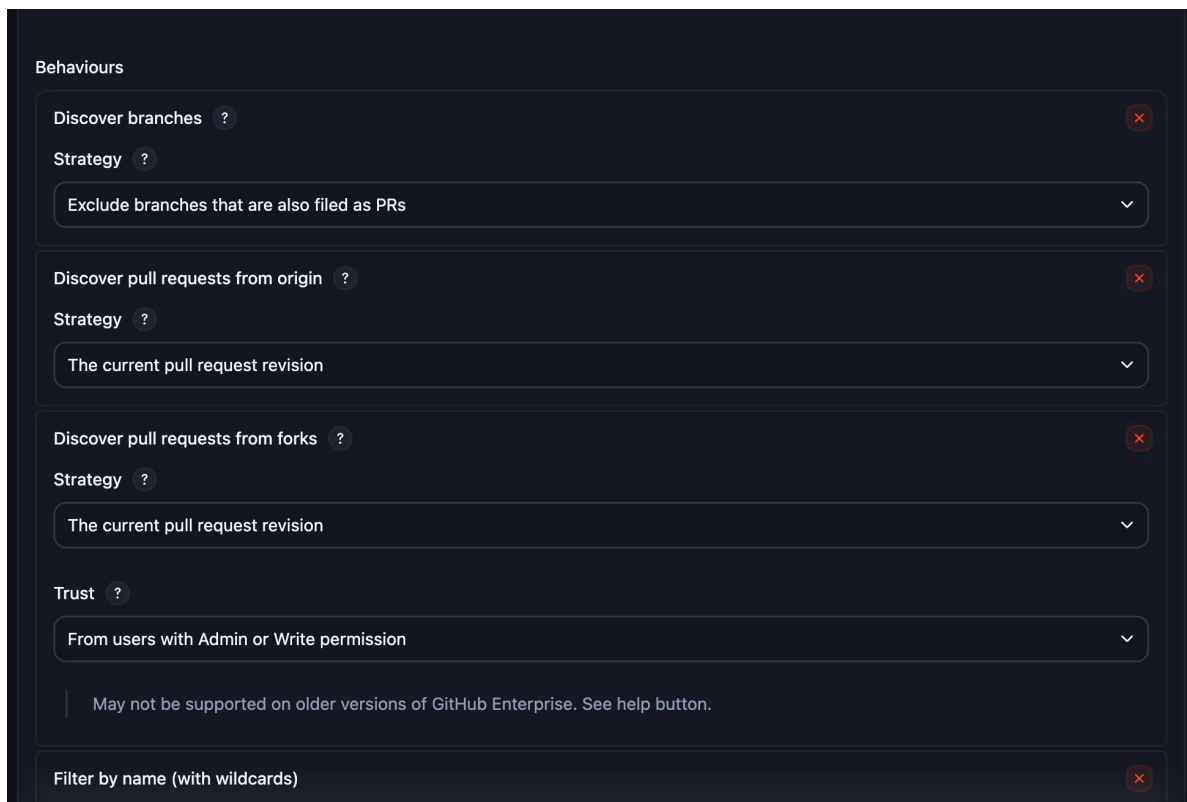
5.1.3 Thiết Lập Multibranch Pipeline

Bảng 9: Cấu hình Multibranch Pipeline Job

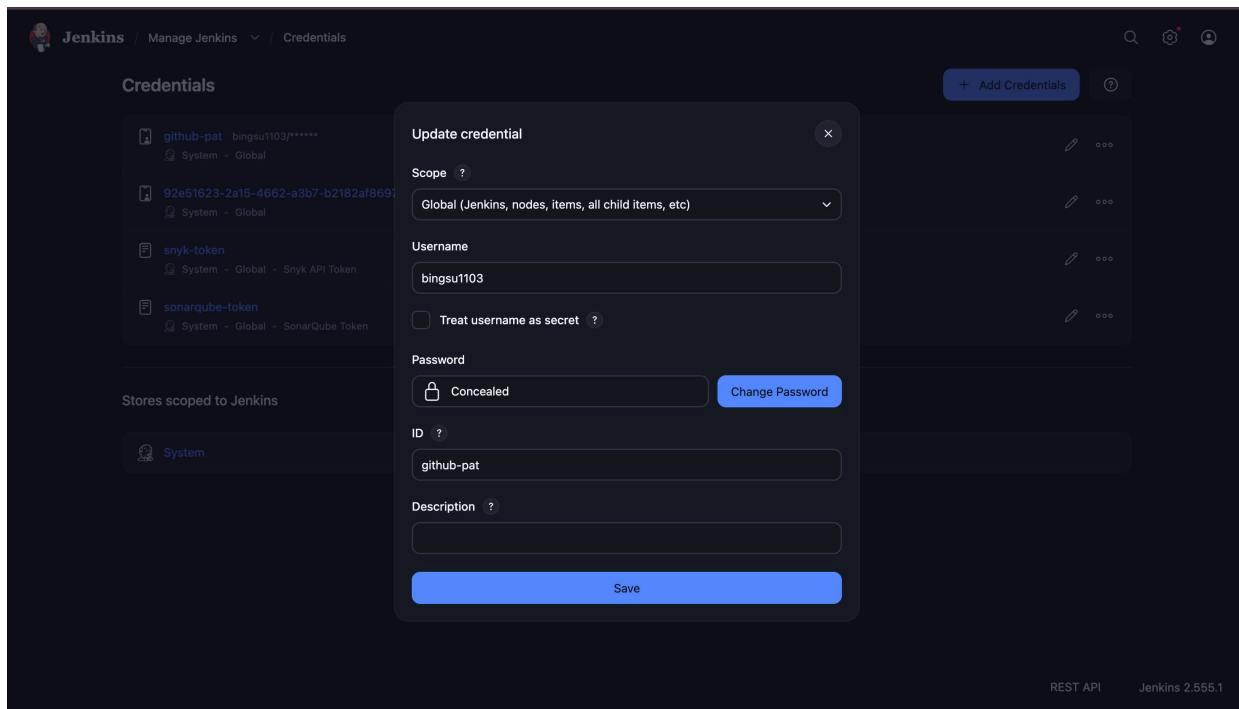
Cấu hình	Giá trị
Tên job	YAS
Loại job	Multibranch Pipeline
Branch Source	GitHub
URL Repository	https://github.com/com-suon-bi-cha/yas.git
Scan interval	2 minute
Đường dẫn Jenkinsfile	Jenkinsfile (tại root)



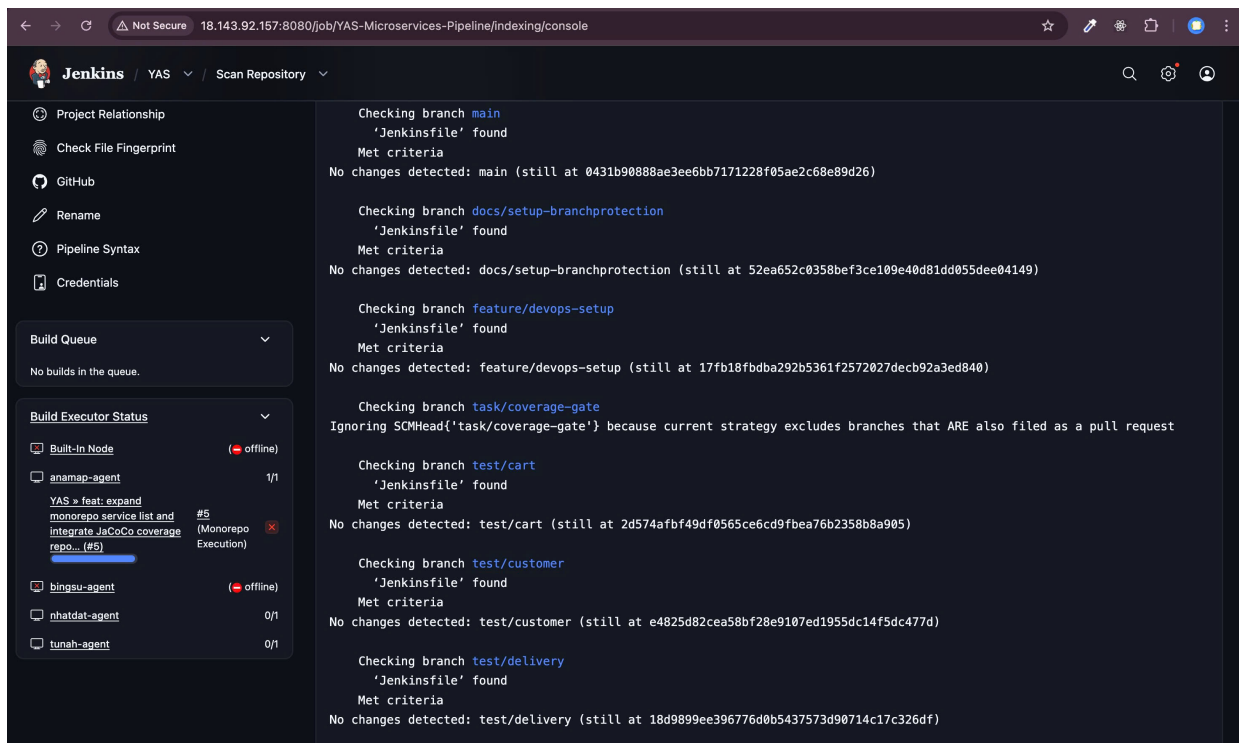
Hình 5: Cấu hình Multibranch Pipeline Job (1)



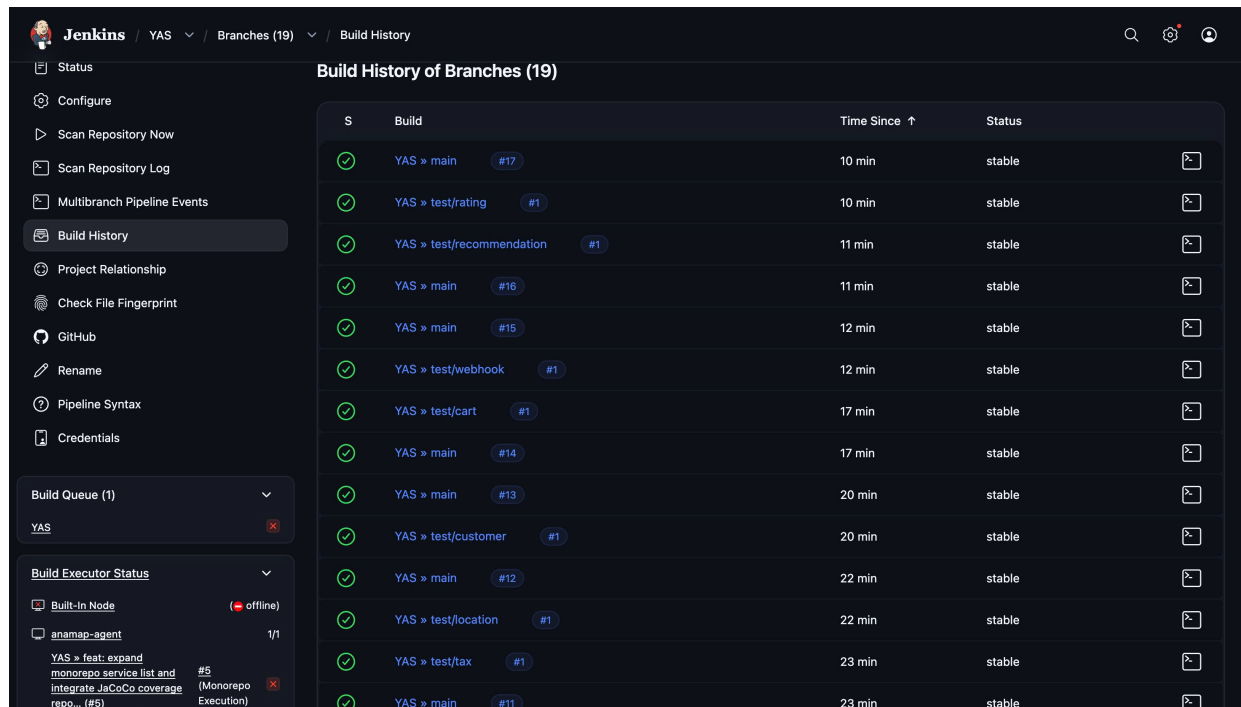
Hình 6: Cấu hình Multibranch Pipeline Job (2)



Hình 7: Cấu hình Personal Access Token cho Multibranch Pipeline



Hình 8: Multibranch Pipeline — danh sách branch được Jenkins phát hiện



Hình 9: Pipeline tự động kích hoạt sau khi push code

5.1.4 Nội Dung Jenkinsfile

Cấu Trúc Pipeline Pipeline gồm 7 stages theo thứ tự:

```

1 pipeline {
2   agent { label 'individual-agent' }
3
4   stages {
5     stage('Pre-check')           { ... }
6     stage('Secret Scanning')     { ... }
7     stage('Monorepo Execution') { ... }
8     stage('Code Quality')        { ... }
9     stage('Quality Gate')        { ... }
10    stage('Coverage Report')     { ... }
11    stage('Dependency Scan')     { ... }
12  }
13 }
```

Listing 2: Cấu trúc Jenkinsfile

Logic Monorepo Execution (Test → Upload → Build) Đây là stage cốt lõi, thực hiện cả 2 giai đoạn **Test** và **Build**. Pipeline sử dụng `git diff` để chỉ xử lý service có thay đổi:

```

1 stage('Monorepo Execution') {
2     steps {
3         script {
4             def changedFiles = sh(
5                 script: 'git diff --name-only HEAD~1 HEAD',
6                 returnStdout: true
7             ).trim().split('\n')
8
9             def services = [
10                 'media', 'product', 'order', 'inventory',
11                 'payment', 'promotion', 'rating', 'delivery',
12                 'sampledata', 'recommendation', 'customer',
13                 'location', 'cart', 'tax', 'search', 'webhook',
14                 'common-library', 'backoffice-bff', 'storefront-bff'
15             ]
16
17             for (service in services) {
18                 if (changedFiles.any { it.startsWith("${service}/") }) {
19                     sh "mvn test -pl ${service} -am" // Test
20                     junit testResults: "${service}/target/surefire-reports/*.xml",
21
22                     allowEmptyResults: true // Upload
23                     sh "mvn package -DskipTests -pl ${service} -am" // Build
24                 }
25             }
26         }
27     }
28 }
    
```

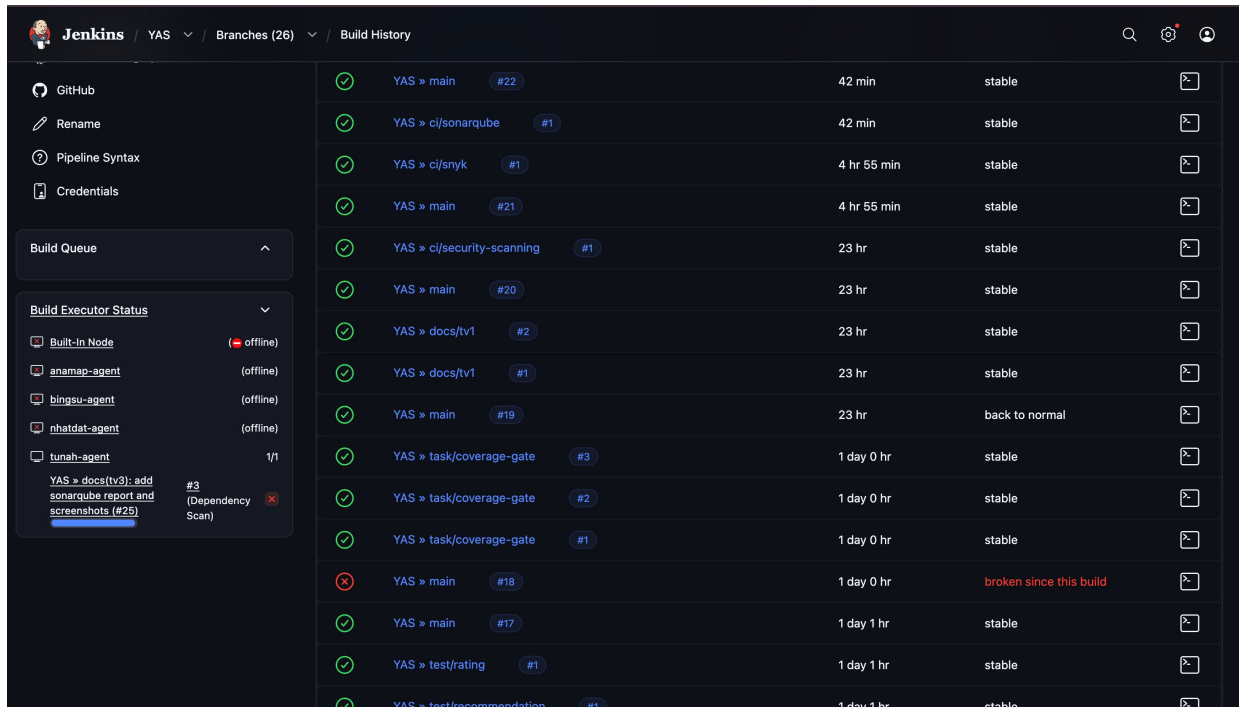
Listing 3: Logic Monorepo Execution trong Jenkinsfile

Giải thích 3 bước:

- `mvn test` — chạy toàn bộ unit test của service, JaCoCo agent tự động thu thập coverage data.

- ### 5.1.5 Kiểm Tra Pipeline Hoạt Động





Hình 11: Lịch sử build trên Jenkins

5.1.6 Cấu Hình Agent Pool (Distributed Builds)

Master node (Built-In Node) của Jenkins chỉ có **2 GB RAM** (instance `t3.small`), không đủ tài nguyên để build/test các service Java lớn trong monorepo YAS. Nhóm triển khai mô hình **Distributed Builds** với một pool gồm 4 agent riêng biệt.

Lợi ích:

- **Giảm tải master node:** Master chỉ đảm nhiệm điều phối (scheduling), toàn bộ workload build/test được chuyển sang agent.
- **Tận dụng tài nguyên cá nhân:** Mỗi thành viên cấu hình máy cá nhân làm agent, chủ động bật agent khi có thay đổi cần chạy pipeline.
- **Song song hóa:** Nhiều pipeline có thể chạy đồng thời trên các agent khác nhau.
- **Môi trường linh hoạt:** Agent có thể chạy trên nhiều kiến trúc khác nhau (aarch64, amd64).

Danh sách agent:

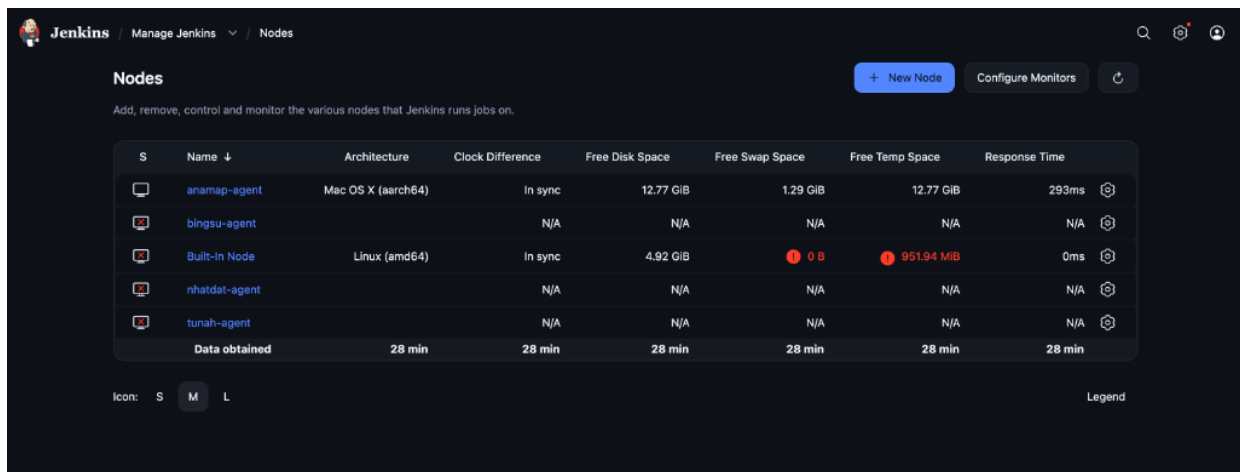
- `anamax-agent`

- bingsu-agent
- nhatdat-agent
- tunah-agent

Tất cả agent được gán chung label `individual-agent` để Jenkins có thể tự động phân phối job đến bất kỳ agent nào đang online.

```
1 pipeline {
2   agent { label 'individual-agent' }
3   // ...
4 }
```

Listing 4: Cấu hình agent label trong Jenkinsfile



S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	anamap-agent	Mac OS X (aarch64)	In sync	12.77 GiB	1.29 GiB	12.77 GiB	293ms
	bingsu-agent		N/A	N/A	N/A	N/A	N/A
	Built-In Node	Linux (amd64)	In sync	4.92 GiB	0 B	951.94 MiB	0ms
	nhatdat-agent		N/A	N/A	N/A	N/A	N/A
	tunah-agent		N/A	N/A	N/A	N/A	N/A
Data obtained		28 min	28 min	28 min	28 min	28 min	28 min

Hình 12: Danh sách Nodes trên Jenkins: 4 agent + Built-In Node

5.2 Branch Protection

5.2.1 Cấu Hình Branch Protection Trên GitHub

Cấu hình tại: GitHub Repository > Settings > Branches > Add branch protection rule

Bảng 10: Các rule Branch Protection đã cấu hình

Rule	Giá trị	Mục đích
Branch name pattern	main	Áp dụng cho nhánh chính
Require a pull request before merging	Bật	Bắt buộc tạo PR, cấm push trực tiếp
Required number of approvals	2	Cần ít nhất 2 thành viên approve
Require status checks to pass	Bật	Jenkins CI phải pass trước khi merge
Require branches to be up to date	Bật	Branch phải được sync với main
Do not allow bypassing above settings	Bật	Admin cũng phải tuân thủ quy tắc

Branch protection rule

Protect your most important branches

[Branch protection rules](#) define whether collaborators can delete or force push to the branch and set requirements for any pushes to the branch, such as passing status checks or a linear commit history.

[Your organization's GitHub Free plan](#) can only enforce rules on its public repositories, like this one.

Branch name pattern *

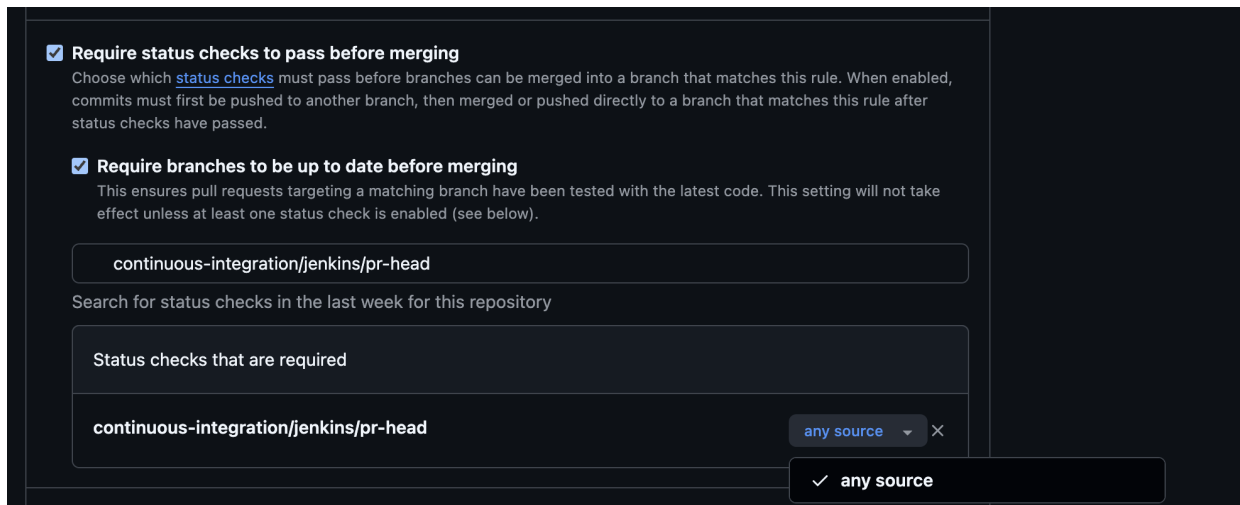
Protect matching branches

☒ **Require a pull request before merging**
 When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

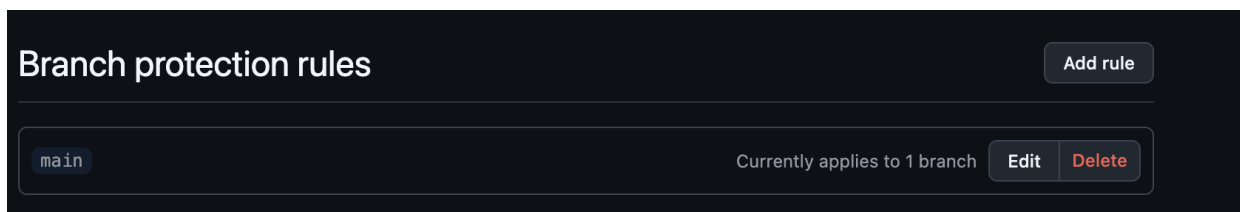
☒ **Require approvals**
 When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

Required number of approvals before merging: 2 ▼

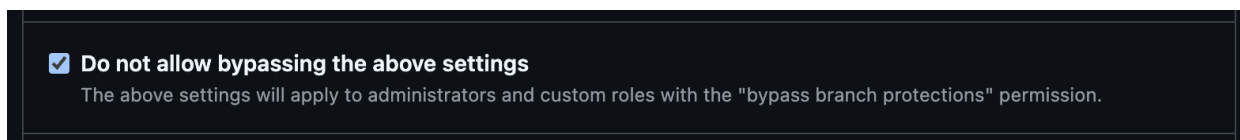
Hình 13: Cấu hình bắt buộc tạo Pull Request và số lượng Approval



Hình 14: Cấu hình bắt buộc Status Checks (Jenkins CI) phải pass



Hình 15: Cấu hình bắt buộc cập nhật nhánh trước khi merge

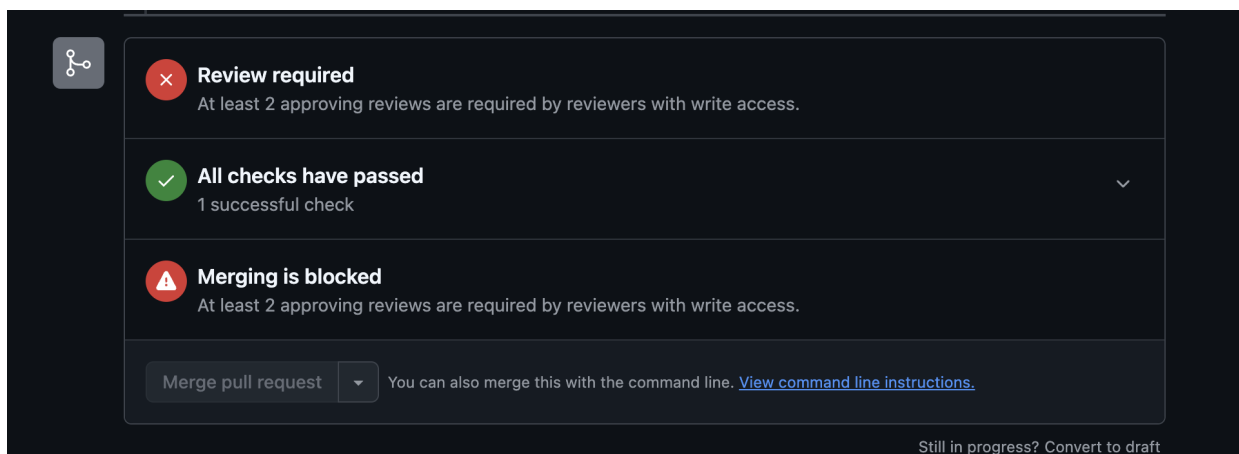


Hình 16: Cấu hình không cho phép Admin lách luật

```

• > echo "Test branch protection" > test-protection.txt
• > git add test-protection.txt
• > git commit -m "test: add dummy file to test branch protection"
[main 449d6f60] test: add dummy file to test branch protection
1 file changed, 1 insertion(+)
create mode 100644 test-protection.txt
ⓧ > git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 333 bytes | 333.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote: This repository moved. Please use the new location:
remote: https://github.com/com-suon-bi-cha/yas.git
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote:
remote: - Changes must be made through a pull request.
remote:
remote: - Required status check "continuous-integration/jenkins/pr-head" is expected.
To https://github.com/NgBaoAnn/yas.git
! [remote rejected] main -> main (protected branch hook declined)
error: failed to push some refs to 'https://github.com/NgBaoAnn/yas.git'
    
```

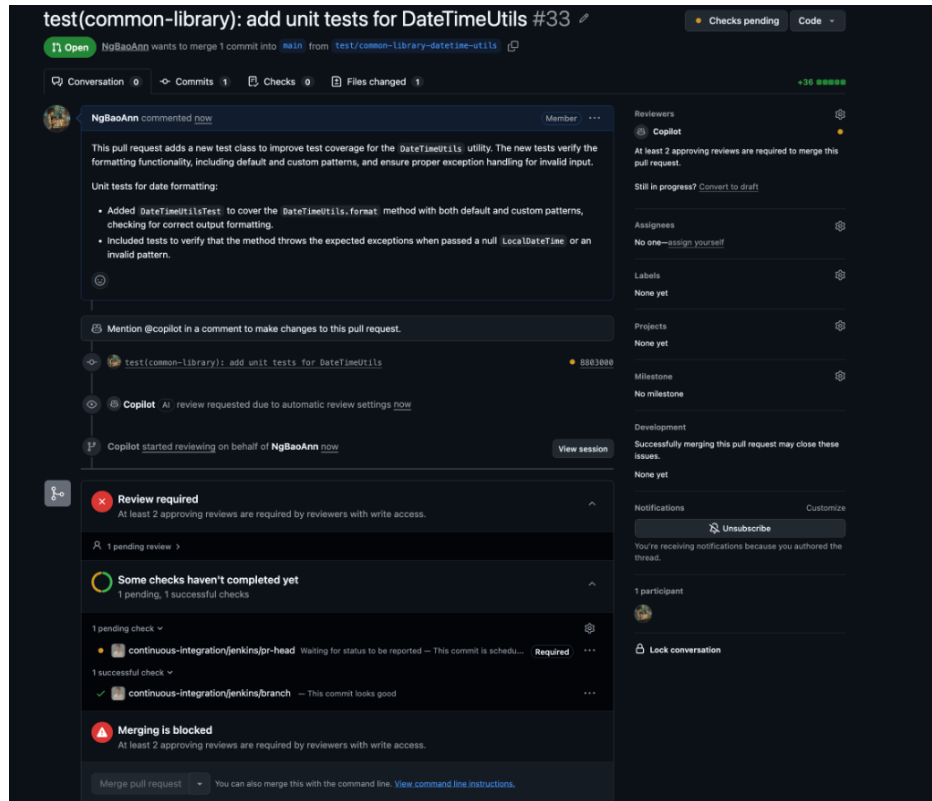
Hình 17: Push trực tiếp vào nhánh main bị từ chối



Hình 18: Pull Request hiển thị yêu cầu 2 lượt approve và CI check phải pass

5.2.2 Pull Request Demo (Trạng Thái Open)

Theo yêu cầu nộp bài, nhóm duy trì ít nhất một PR ở trạng thái Open trên GitHub.



Hình 19: Pull Request #33 đang ở trạng thái Open, yêu cầu 2 approvals và CI pass

5.3 Security Scanning

5.3.1 Tích Hợp Gitleaks — Quét Secret Bị Lộ

Gitleaks là công cụ quét mã nguồn để phát hiện các thông tin nhạy cảm bị commit nhầm vào repository, bao gồm API key, mật khẩu, token truy cập và các credential khác.

```

1 stage('Secret Scanning') {
2     steps {
3         sh '''
4             gitleaks detect --source . \
5                 --config gitleaks.toml \
6                 --report-format json \
7                 --report-path gitleaks-report.json \
8                 --exit-code 1
9         '''
10    }
11    post {

```

```

12     always {
13         archiveArtifacts artifacts: 'gitleaks-report.json',
14             allowEmptyArchive: true
15     }
16 }
17 }
```

Listing 5: Stage Secret Scanning trong Jenkinsfile

Pipeline sẽ dừng lại (FAIL) ngay tại stage này nếu Gitleaks phát hiện secret bị lộ.

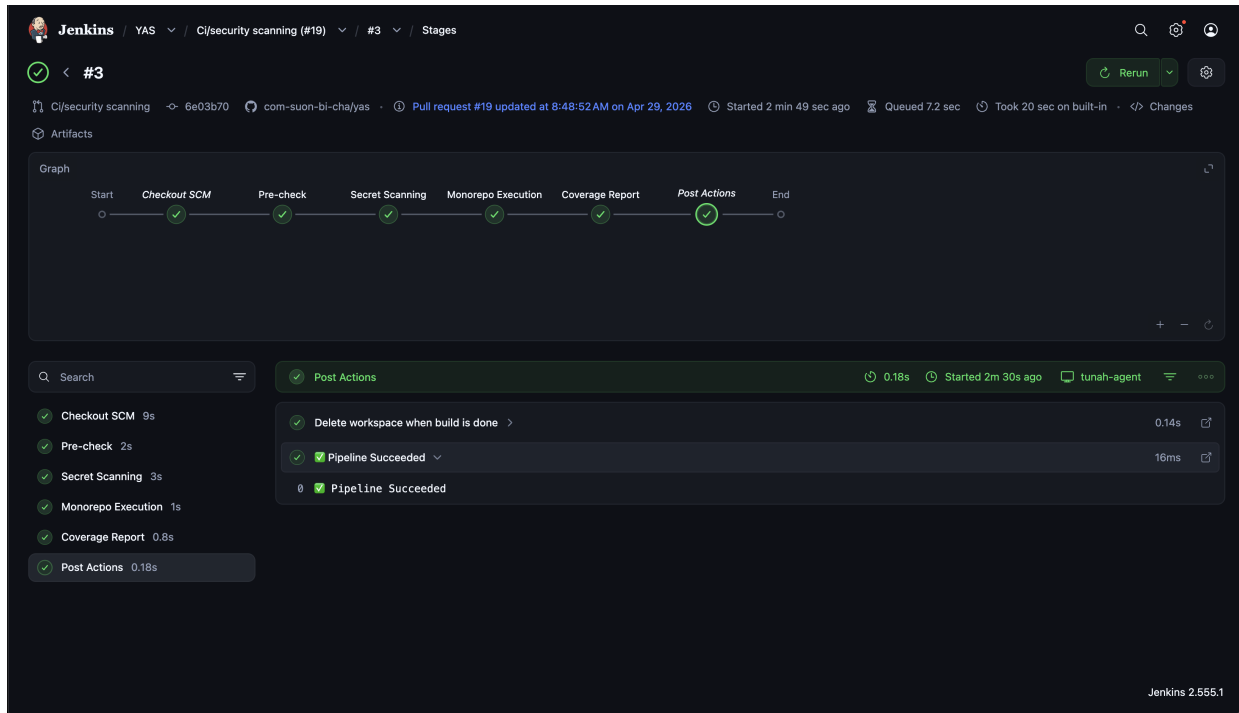
Xử Lý False Positive Sau lần chạy đầu tiên, Gitleaks phát hiện **13 findings** trong lịch sử commit — tất cả đều là Keycloak client secret từ **upstream repository** (nashtech-garage/yas), không phải secret thật của nhóm. Xử lý bằng cách thêm **commits** vào **allowlist** trong **gitleaks.toml**:

```

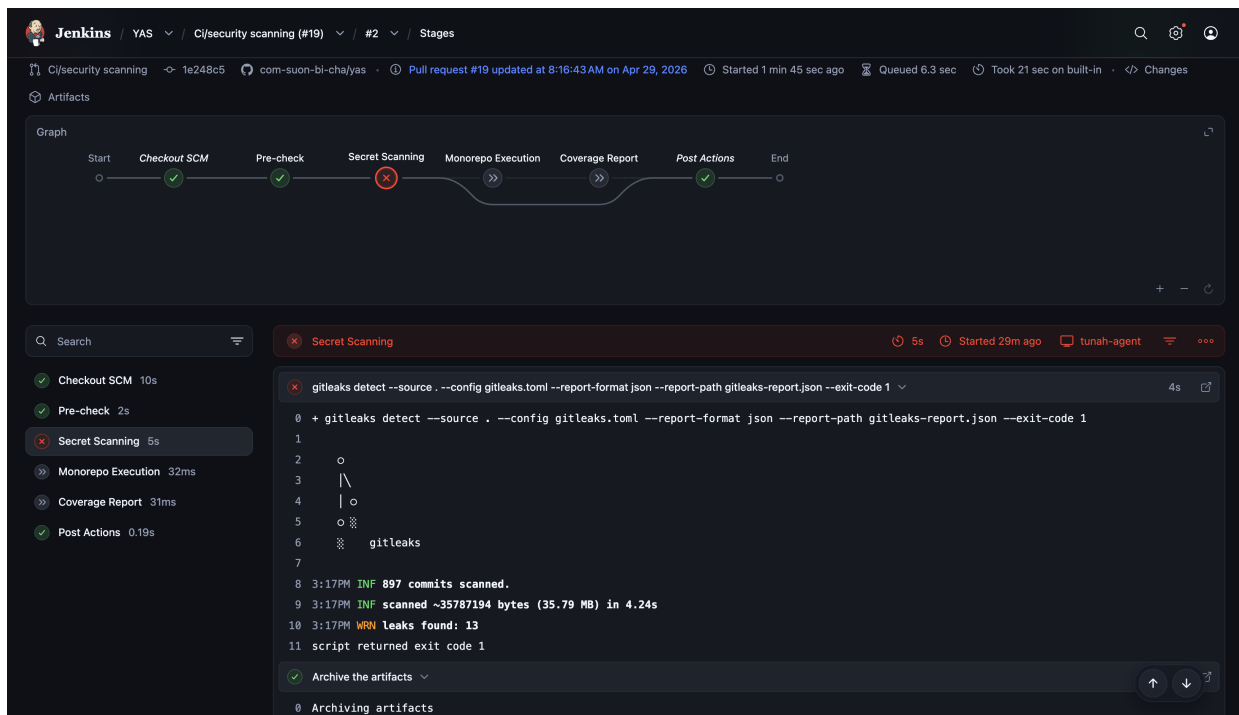
1 [allowlist]
2 description = "global allow list"
3 paths = [ '''test-realm.json''', '''realm-export''', ... ]
4 commits = [
5     "af2c9421030761ec4eccb0994a6c576592be113b",
6     "8dc5e08456e8fa8c970b7b0cadcffdc15d77e39",
7     ...
8 ]
```

Listing 6: Cấu hình allowlist trong gitleaks.toml

Sử dụng **commits** allowlist thay vì **paths** để đảm bảo chính xác — chỉ bỏ qua đúng các commit từ upstream, không bỏ qua các secret thật có thể xuất hiện trong tương lai.



Hình 20: Stage Secret Scanning chạy thành công (không phát hiện secret)



Hình 21: Pipeline thất bại khi Gitleaks phát hiện secret

5.3.2 Tích Hợp SonarQube — Phân Tích Chất Lượng Code

Bảng 11: Thông số SonarQube Server

Thông số	Giá trị
Phương thức triển khai	Docker (deploy trên Jenkins server)
Phiên bản SonarQube	v26.4.0.121862 (Community Build)
Địa chỉ truy cập	http://18.140.115.86:9000
Project Key	yas

Cài Đặt SonarQube Server

Các Bước Kết Nối Jenkins Với SonarQube

1. Cài plugin **SonarQube Scanner** trên Jenkins.
2. Tạo token trên SonarQube: My Account > Security > Generate Tokens.
3. Lưu token vào Jenkins Credentials với ID sonarqube-token.
4. Thêm SonarQube server: Manage Jenkins > System > SonarQube servers.
5. Tạo webhook từ SonarQube về Jenkins: Administration > Configuration > Webhooks.

```

1 stage('Code Quality') {
2     steps {
3         withSonarQubeEnv('SonarQube') {
4             sh 'mvn sonar:sonar -Dsonar.projectKey=yas \
5                 -Dsonar.java.binaries=.'
6         }
7     }
8 }
9 stage('Quality Gate') {
10    steps {
11        timeout(time: 5, unit: 'MINUTES') {
12            waitForQualityGate abortPipeline: true
13        }
14    }
15 }
```

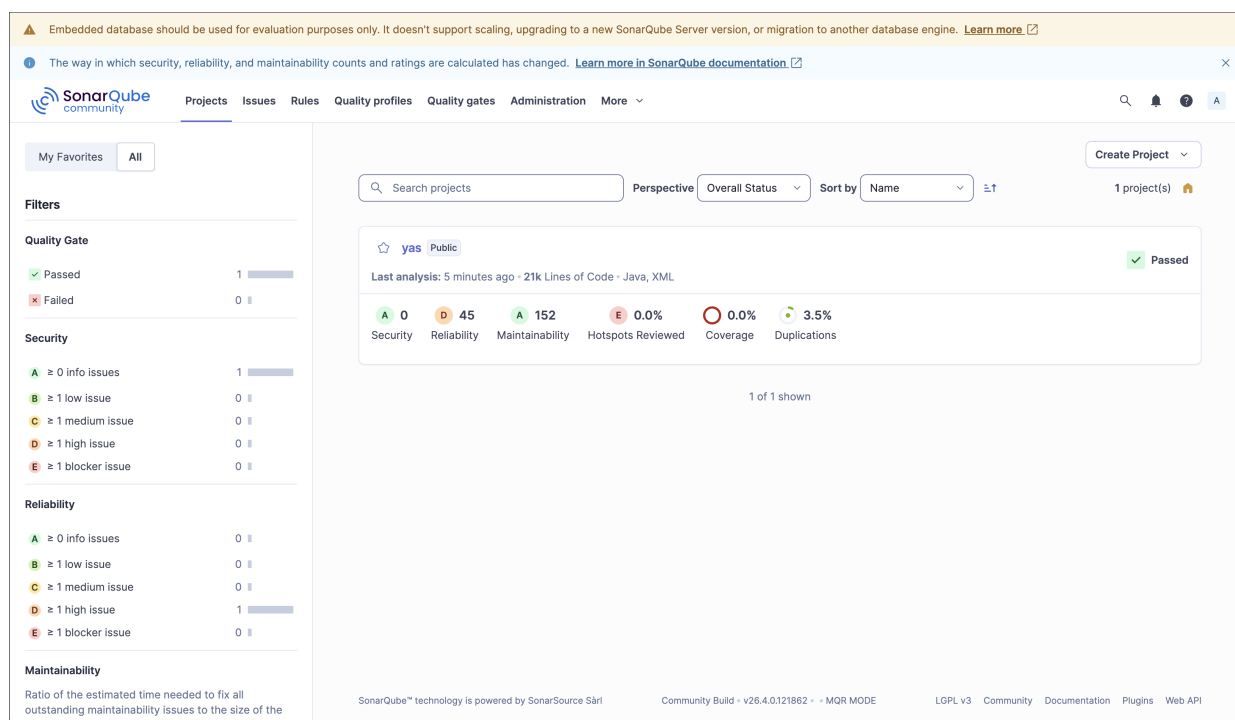
Listing 7: Stage Code Quality và Quality Gate trong Jenkinsfile

Kết Quả Phân Tích SonarQube phân tích toàn bộ monorepo `yas` với **21k Lines of Code**. Kết quả Quality Gate: **Passed**.

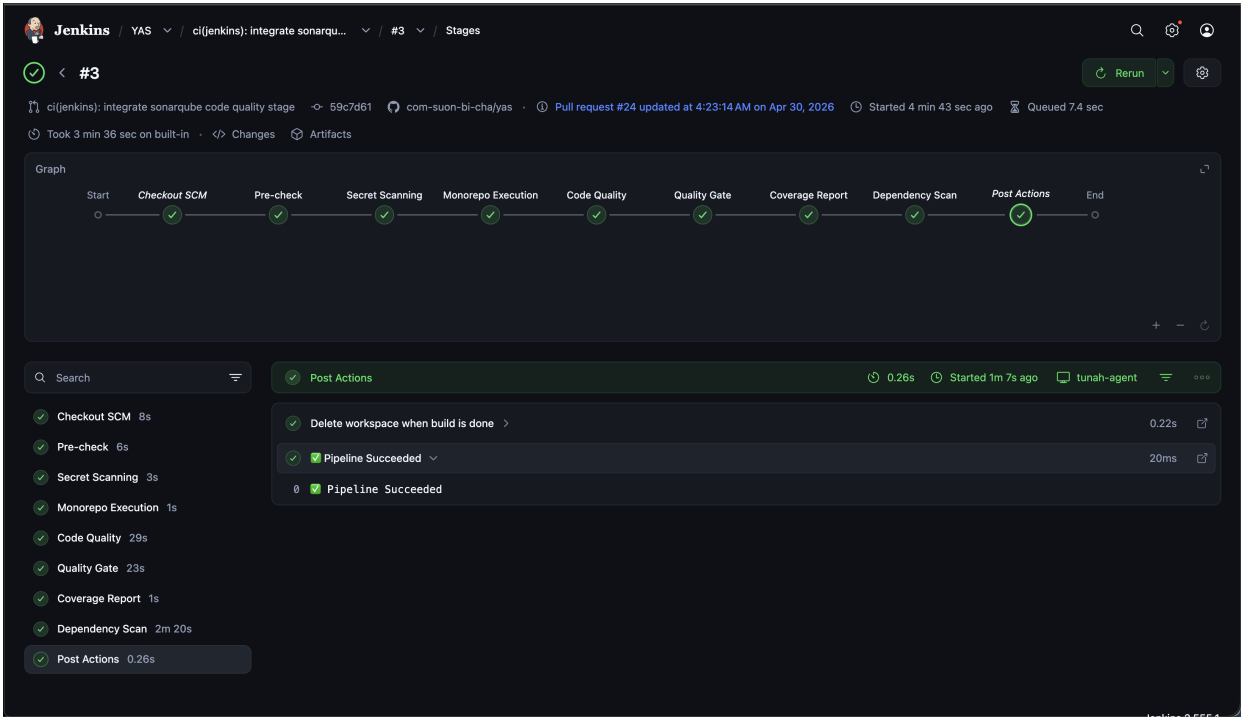
Bảng 12: Kết quả phân tích SonarQube

Chỉ số	Kết quả
Security	0 issues
Reliability	45 issues
Maintainability	152 issues
Coverage	0.0% (chưa tích hợp JaCoCo report)
Duplications	3.5%

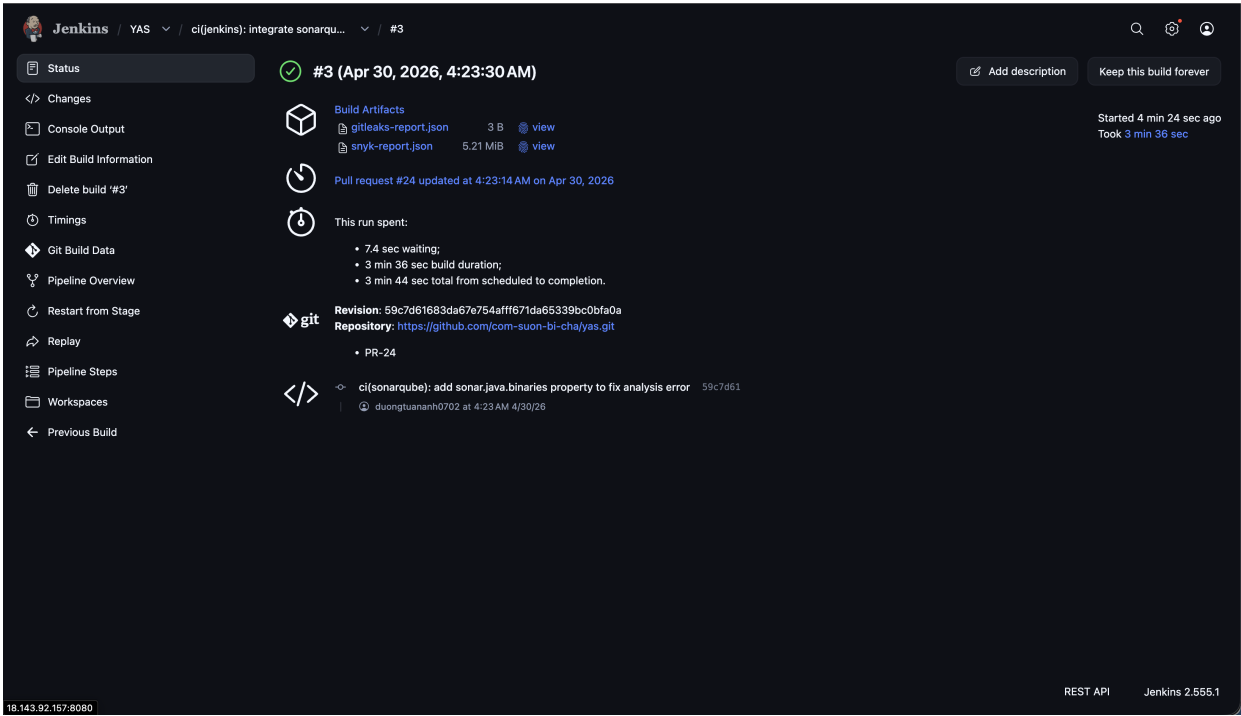
Lưu ý: Coverage hiển thị 0.0% trên SonarQube vì pipeline chưa cấu hình `sonar.coverage.jacoco.xmlReport`. Độ phủ test thực tế được đo và enforce bằng JaCoCo plugin trực tiếp trên Jenkins (xem phần Coverage Gate).



Hình 22: SonarQube Dashboard: project `yas` với Quality Gate Passed



Hình 23: Jenkins pipeline: stage Code Quality và Quality Gate chạy thành công



Hình 24: Jenkins build summary: pipeline hoàn thành thành công

5.3.3 Tích Hợp Snyk — Quét Lỗ Hổng Dependency

Bảng 13: Thông số cấu hình Snyk

Thông số	Giá trị
Tài khoản Snyk	https://app.snyk.io
Phương thức xác thực	API Token (lưu trong Jenkins Credentials với ID snyk-token)
Phương thức tích hợp	Snyk CLI (snyk test + snyk monitor)

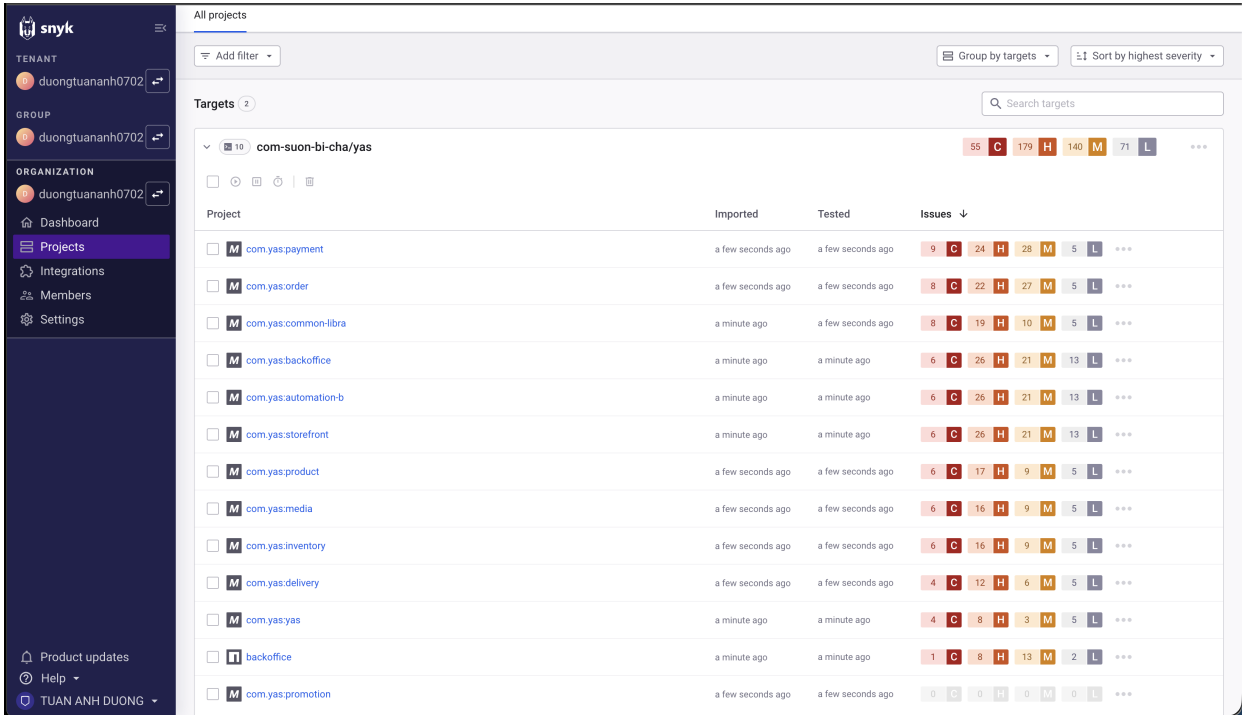
```

1 stage('Dependency Scan') {
2     steps {
3         withCredentials([string(credentialsId: 'snyk-token',
4                                 variable: 'SNYK_TOKEN')]) {
5             sh 'snyk auth $SNYK_TOKEN'
6             sh 'snyk test --all-projects --json > snyk-report.json || true'
7             sh 'snyk monitor --all-projects || true'
8         }
9     }
10 }
    
```

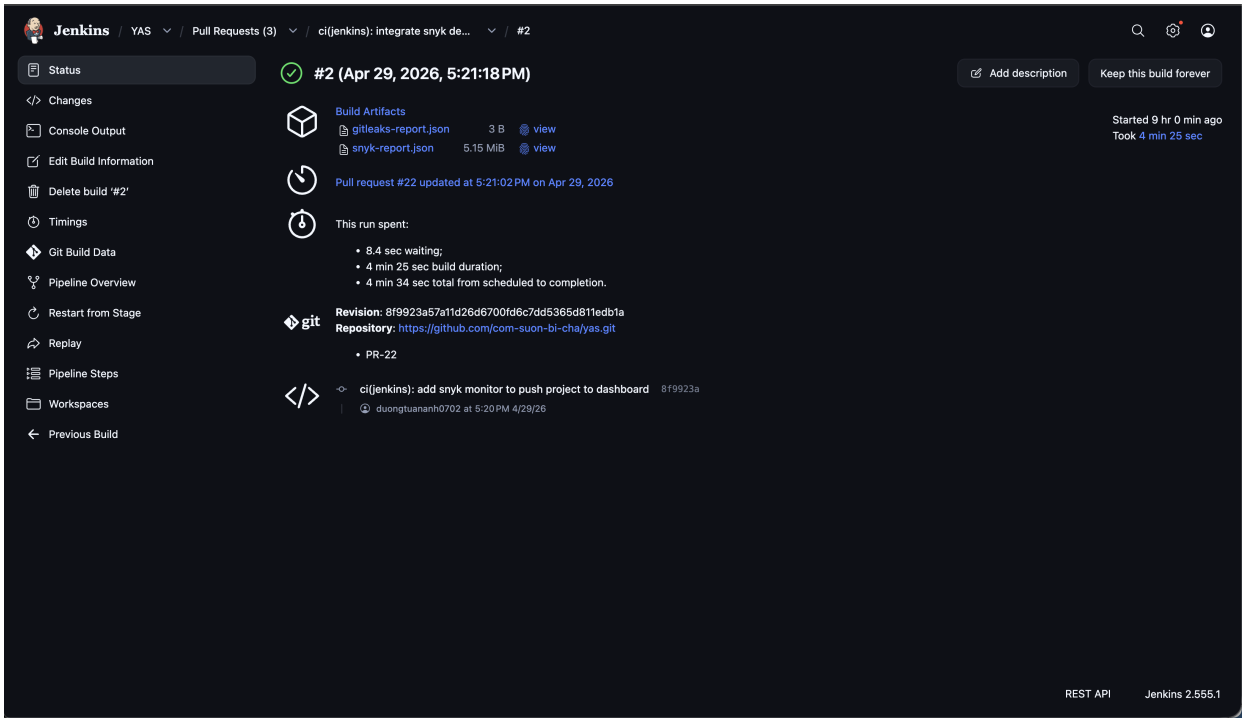
Listing 8: Stage Dependency Scan trong Jenkinsfile

`snyk test` quét và báo cáo vulnerability. `snyk monitor` đẩy kết quả lên Snyk dashboard. Cả hai dùng `|| true` để pipeline không FAIL khi phát hiện vulnerability mức thấp.

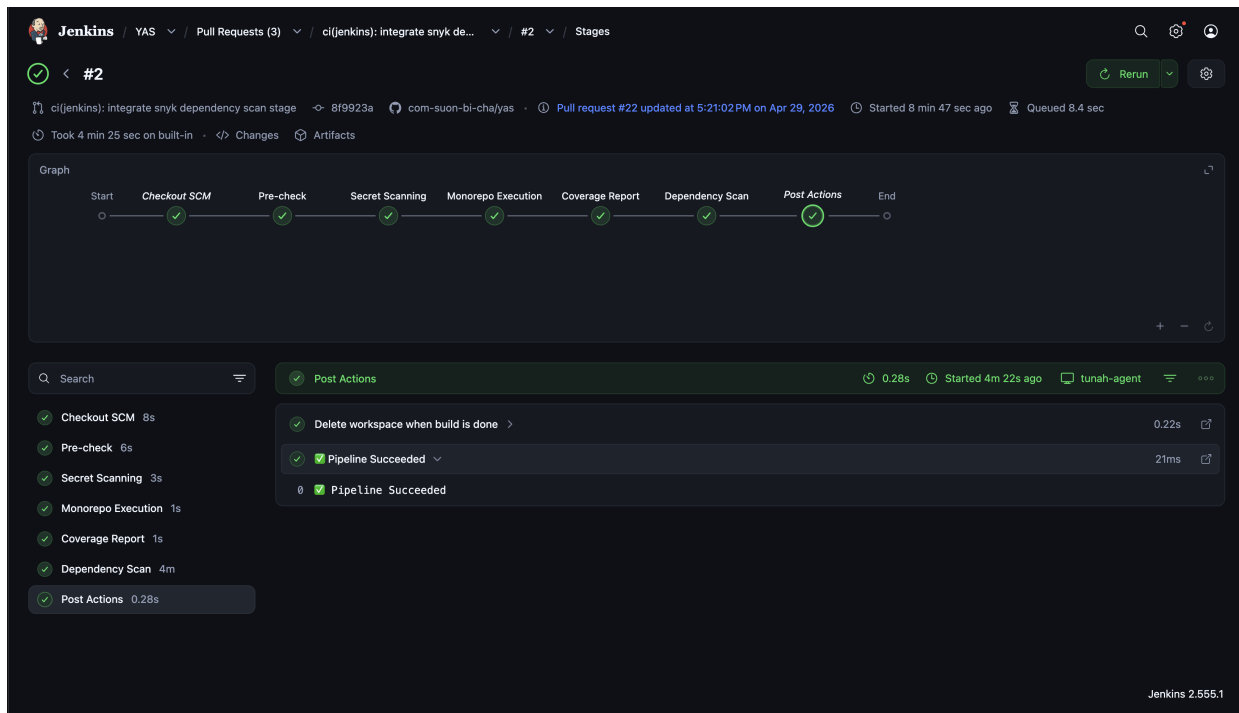
Snyk phát hiện tổng cộng **55 Critical, 179 High, 140 Medium, 71 Low** trên tất cả các module (lỗ hổng trong thư viện bên thứ ba).



Hình 25: Snyk Dashboard: danh sách project và vulnerability được phát hiện



Hình 26: Stage Dependency Scan trong Jenkins pipeline chạy thành công



Hình 27: Toàn bộ pipeline chạy thành công với đầy đủ Security Stages

5.4 Coverage Gate

JaCoCo (Java Code Coverage) đo lường độ phủ của unit test trên mã nguồn. Coverage Gate là cơ chế tự động làm thất bại pipeline khi độ phủ thấp hơn ngưỡng quy định. Nhóm đặt ngưỡng là **70% instruction coverage**.

5.4.1 Cấu Hình Trong pom.xml

Thêm cấu hình check goal vào JaCoCo plugin để pipeline tự động fail khi coverage không đạt:

```

1 <plugin>
2   <groupId>org.jacoco</groupId>
3   <artifactId>jacoco-maven-plugin</artifactId>
4   <executions>
5     <execution>
6       <id>prepare-agent</id>
7       <goals><goal>prepare-agent</goal></goals>
8     </execution>
9     <execution>
10      <id>report</id>

```

```

11         <goals><goal>report</goal></goals>
12     </execution>
13     <execution>
14         <id>check</id>
15         <goals><goal>check</goal></goals>
16         <configuration>
17             <rules>
18                 <rule>
19                     <element>BUNDLE</element>
20                     <limits>
21                         <limit>
22                             <counter>INSTRUCTION</counter>
23                             <value>COVEREDRATIO</value>
24                             <minimum>0.70</minimum>
25                         </limit>
26                     </limits>
27                 </rule>
28             </rules>
29         </configuration>
30     </execution>
31 </executions>
32 </plugin>
    
```

Listing 9: Cấu hình JaCoCo Coverage Gate trong pom.xml

Khi coverage < 70%, lệnh `mvn verify` hoặc `mvn jacoco:check` sẽ trả về BUILD FAILURE.

5.4.2 Cấu Hình Stage Trong Jenkinsfile

```

1 stage('Coverage Report') {
2     steps {
3         script {
4             for (service in services) {
5                 if (changedFiles.any { it.startsWith("${service}/") }) {
6                     jacoco(
7                         execPattern: "${service}/target/jacoco.exec",
8                         classPattern: "${service}/target/classes",
9                         sourcePattern: "${service}/src/main/java",
    
```

```

10         minimumInstructionCoverage: '70',
11         changeBuildStatus: true
12     )
13 }
14 }
15 }
16 }
17 }
    
```

Listing 10: Stage Coverage Report trong Jenkinsfile

Tham số `changeBuildStatus: true` cho phép plugin JaCoCo trên Jenkins tự động đổi trạng thái build sang FAILURE nếu coverage không đạt ngưỡng.

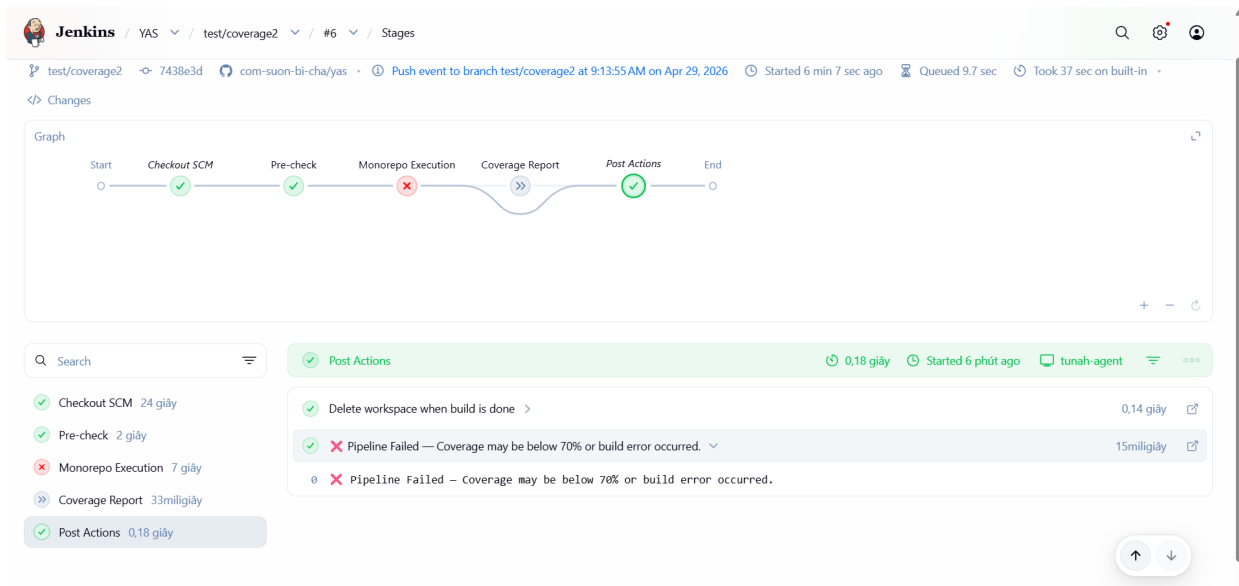
5.4.3 Hình Ảnh Minh Chứng

```

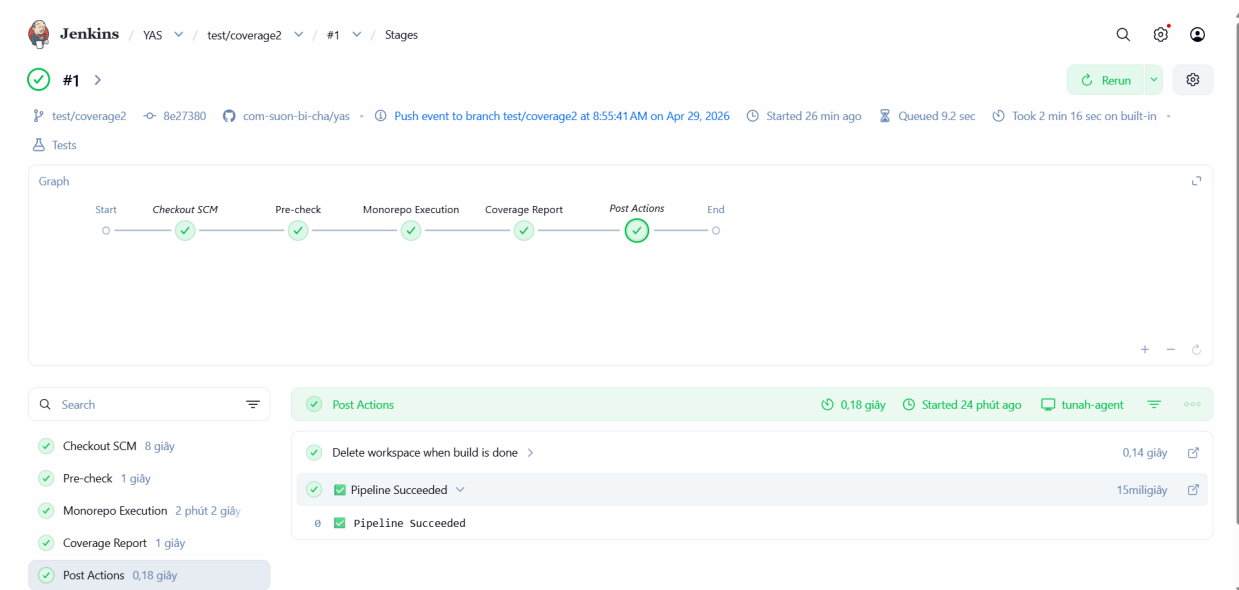
stage('Coverage Report') {
    steps {
        script {
            def changedFiles = sh(script: 'git diff --name-only HEAD~1 HEAD', returnStdout: true).trim().split('\n')
            def services = [
                'media', 'product', 'order', 'inventory', 'payment', 'promotion',
                'rating', 'delivery', 'sampledata', 'recommendation', // TV2 (10 modules)
                'customer', 'location', 'cart', 'tax', 'search', 'webhook', // TV4 (6 modules)
                'common-library', 'backoffice-bff', 'storefront-bff', 'payment-paypal' // Other modules
            ]

            for (service in services) {
                if (changedFiles.any { it.startsWith("${service}/") }) {
                    echo "Publishing JaCoCo report for: ${service}"
                    jacoco(
                        execPattern: "${service}/target/jacoco.exec",
                        classPattern: "${service}/target/classes",
                        sourcePattern: "${service}/src/main/java",
                        exclusionPattern: '**/config/**,**/exception/**,**/constants/**,**/Application.class',
                        minimumInstructionCoverage: '70',
                        minimumBranchCoverage: '0',
                        changeBuildStatus: true
                    )
                    // Explicitly fail if JaCoCo marked build as UNSTABLE
                    if (currentBuild.result == 'UNSTABLE' || currentBuild.result == 'FAILURE') {
                        error("Coverage gate FAILED for ${service}: Instruction coverage is below 70%!")
                    }
                }
            }
        }
    }
}
    
```

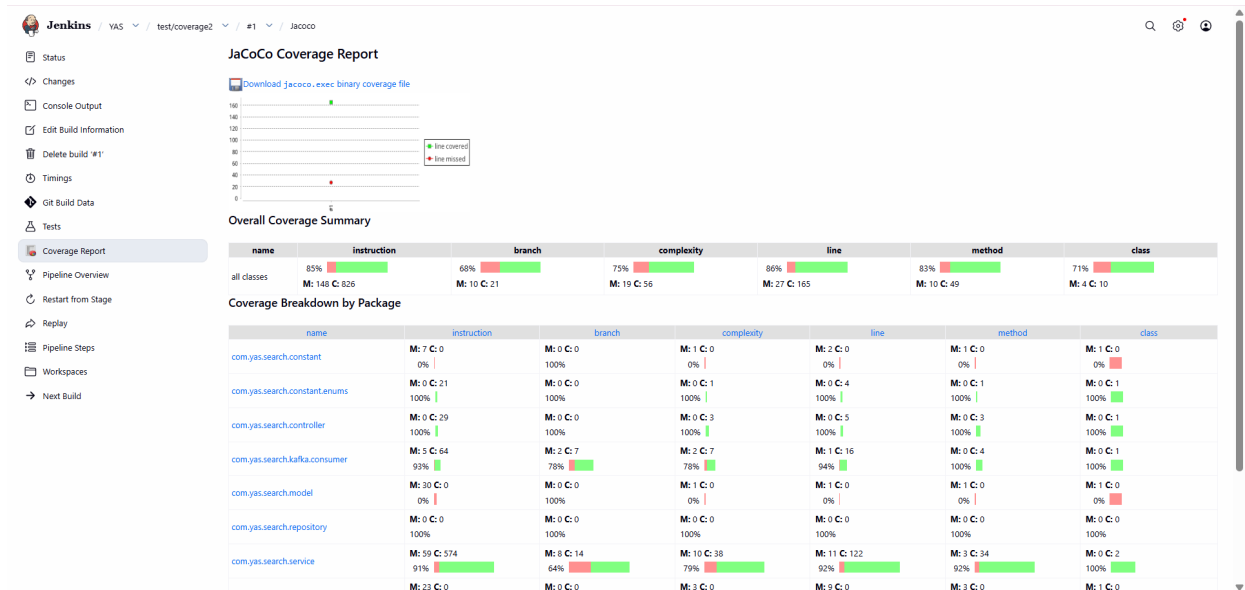
Hình 28: Cấu hình JaCoCo Coverage Gate trong file pom.xml



Hình 29: Pipeline thất bại khi coverage dưới ngưỡng 70%



Hình 30: Pipeline thành công sau khi bổ sung đủ unit test



Hình 31: Báo cáo JaCoCo Coverage hiển thị trong giao diện Jenkins

6 Unit Test — Chi Tiết Từng Module

6.1 Hướng Dẫn Chung Chạy Test

Do project sử dụng cấu trúc monorepo với thuộc tính `${revision}`, lệnh phải chạy từ bên trong thư mục module tương ứng:

```
1 cd /path/to/yas/<module>
2 ./mvnw -f ../pom.xml test -pl <module> -am
3 ./mvnw -f ../pom.xml test jacoco:report -pl <module> -am
4 open target/site/jacoco/index.html
```

Listing 11: Lệnh chạy test và tạo report JaCoCo

6.2 Module media

Branch: `test/media`

Bảng 14: Danh sách file test — module media

File Test	Lớp được kiểm thử	Test case
MediaControllerTest.java	MediaController (5 endpoints)	8
MediaServiceUnitTest.java	MediaService	13
FileSystemRepositoryTest.java	FileSystemRepository	4
StringUtilsTest.java	StringUtils	11
FileTypeValidatorTest.java	FileTypeValidator	6
Tổng		42

Lưu Ý Kỹ Thuật: Annotation `@WebMvcTest` cần loại trừ `OAuth2ResourceServerAutoConfiguration` để tránh lỗi load `ApplicationContext` trong môi trường test:

```

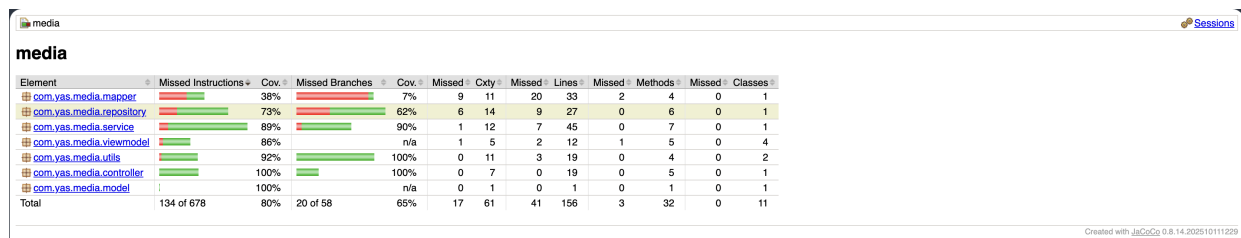
1 @WebMvcTest(controllers = MediaController.class,
2     excludeAutoConfiguration = OAuth2ResourceServerAutoConfiguration.class)
3 @AutoConfigureMockMvc(addFilters = false)
4 class MediaControllerTest { ... }

```

Listing 12: Cấu hình `@WebMvcTest` cho `MediaController`

Bảng 15: Kết quả Coverage — module media (Tổng: 80%)

Package	Instructions	Branches
controller	100%	100%
service	89%	90%
utils	92%	100%
viewmodel	86%	n/a
repository	73%	62%
model	100%	n/a
mapper	38%	7%
Tổng	80%	65%



Hình 32: Báo cáo JaCoCo Coverage — module media (80%)

6.3 Module product

Branch: test/product

Bảng 16: Danh sách file test — module product

File Test	Lớp được kiểm thử	Test case
ProductService*Test.java (10 file)	ProductService	nhiều
CategoryServiceTest.java	CategoryService	nhiều
MediaServiceTest.java	MediaService	4
ProductConverterTest.java	ProductConverter	nhiều
Tổng		178

Bảng 17: Kết quả Coverage — module product (Tổng: 71%)

Package	Instructions	Branches
controller	87%	58%
service	64%	46%
validation	93%	50%
Tổng	71%	47%

product

product

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.yas.product.service	64%	46%	164	309	380	1,172	84	189	0	10		
com.yas.product.viewmodel.product	78%	40%	15	40	23	95	10	35	8	28		
com.yas.product.controller	87%	58%	17	103	16	266	12	97	0	10		
com.yas.product.utils	35%	100%	6	8	11	14	6	7	3	4		
com.yas.product.viewmodel.category	83%	50%	2	8	3	14	0	6	0	4		
com.yas.product.viewmodel.productoption	73%	n/a	2	8	2	14	2	8	2	6		
com.yas.product.viewmodel.error	0%	n/a	2	2	3	3	2	2	1	1		
com.yas.product.viewmodel.productattribute	90%	100%	2	17	2	28	2	16	2	11		
com.yas.product.validation	93%	50%	1	4	0	4	0	3	0	1		
com.yas.product.viewmodel.producttemplate	100%	n/a	0	8	0	12	0	8	0	6		
com.yas.product.viewmodel.brand	100%	n/a	0	5	0	9	0	5	0	3		
com.yas.product.viewmodel	100%	n/a	0	2	0	2	0	2	0	2		
Total	2,238 of 7,797	71%	142 of 272	47%	211	514	440	1,633	118	378	16	86

Hình 33: Báo cáo JaCoCo Coverage — module product (71%)

6.4 Module order

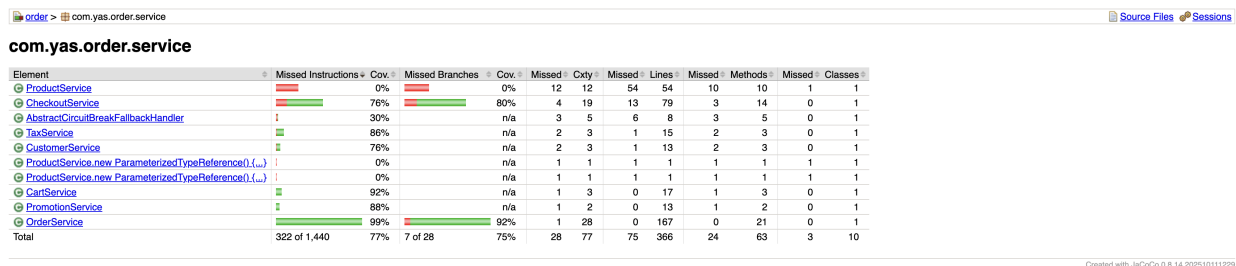
Branch: test/order

Bảng 18: Danh sách file test — module order

File Test	Lớp được kiểm thử	Test case
OrderServiceCreateTest.java	createOrder	1
OrderServiceGetTest.java	getOrderWithItemsById, getAllOrder, ...	11
OrderServiceStatusTest.java	updateOrderPaymentStatus, rejectOrder, ...	7
OrderServiceOtherTest.java	isOrderCompleted, exportCsv	4
CheckoutServiceTest.java	CheckoutService	8
Tổng		31+

Bảng 19: Kết quả Coverage — module order (Tổng: 76%)

Package	Instructions	Branches
service	77%	75%
specification	43%	34%
mapper	76%	44%
Tổng	76%	47%



Hình 34: Báo cáo JaCoCo Coverage — module order (76%)

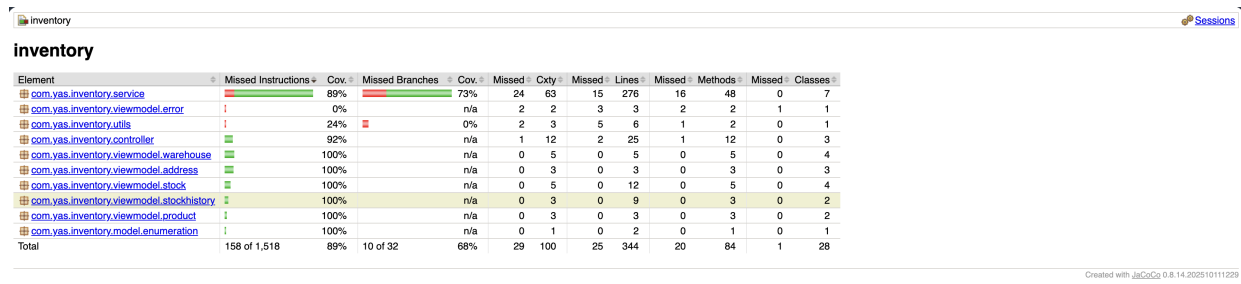
6.5 Module inventory

Branch: test/inventory

Bảng 20: Danh sách file test — module inventory

File Test	Lớp được kiểm thử	Test case
WarehouseServiceTest.java	WarehouseService	9
StockServiceTest.java	StockService	6
StockHistoryServiceTest.java	StockHistoryService	2
LocationServiceTest.java	LocationService	4
ProductServiceTest.java	ProductService	3
Tổng		24+

Kết Quả Coverage: Instructions **89%** | Branches **70%** — Đạt $\geq 70\%$



Hình 35: Báo cáo JaCoCo Coverage — module inventory (89%)

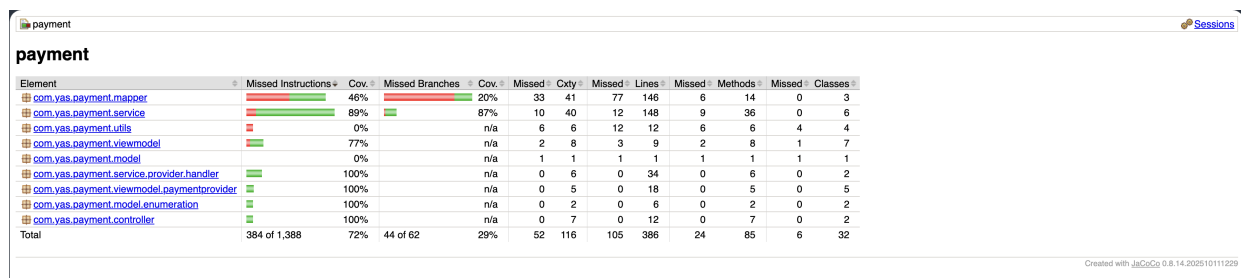
6.6 Module payment

Branch: test/payment

Bảng 21: Danh sách file test — module payment

File Test	Lớp được kiểm thử	Test case
PaymentControllerTest.java	PaymentController	3
PaymentProviderControllerTest.java	PaymentProviderController	3
PaymentProviderServiceTest.java	PaymentProviderService	7
OrderServiceTest.java	OrderService	2
PaypalHandlerTest.java	PaypalHandler	3
PaymentServiceTest.java	PaymentService	2
MediaServiceTest.java	MediaService	2
Tổng		22

Kết Quả Coverage: Instructions **72.33%** — Đạt $\geq 70\%$



Hình 36: Báo cáo JaCoCo Coverage — module payment (72.33%)

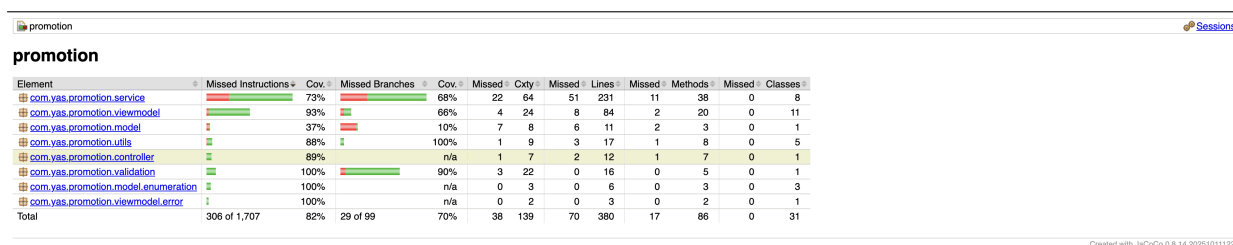
6.7 Module promotion

Branch: test/promotion

Bảng 22: Danh sách file test — module promotion

File Test	Lớp được kiểm thử	Test case
PromotionControllerTest.java	PromotionController	11
PromotionServiceTest.java	PromotionService	14
ProductServiceTest.java	ProductService	5
PromotionValidatorTest.java	PromotionValidator	8
ErrorVmTest.java	ErrorVm	2
AuthenticationUtilsTest.java	AuthenticationUtils	3
MessagesUtilsTest.java	MessagesUtils	2
Tổng		51

Kết Quả Coverage: Instructions **82.07%** — Đạt $\geq 70\%$



Hình 37: Báo cáo JaCoCo Coverage — module promotion (82.07%)

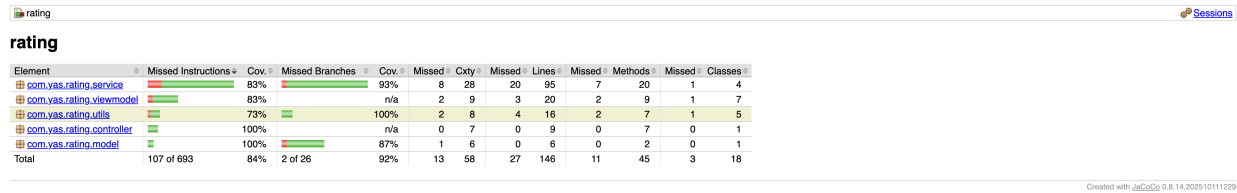
6.8 Module rating

Branch: test/rating

Bảng 23: Danh sách file test — module rating

File Test	Lớp được kiểm thử	Test case
RatingServiceTest.java	RatingService	14
OrderServiceTest.java	OrderService	2
AuthenticationUtilsTest.java	AuthenticationUtils	2
ConstantsTest.java	Constants	1
MessagesUtilsTest.java	MessagesUtils	2
RatingTest.java	Rating (model)	2
Tổng		23

Kết Quả Coverage: Instructions **84.56%** — Đạt $\geq 70\%$



Hình 38: Báo cáo JaCoCo Coverage — module rating (84.56%)

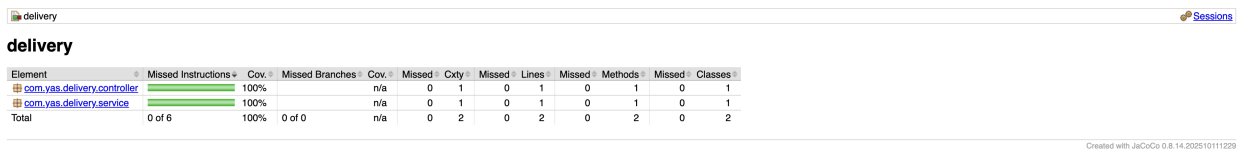
6.9 Module delivery

Branch: test/delivery

Bảng 24: Danh sách file test — module delivery

File Test	Lớp được kiểm thử	Test case
DeliveryApplicationTests.java	DeliveryApplication	1
DeliveryControllerTest.java	DeliveryController	1
DeliveryServiceTest.java	DeliveryService	1
Tổng		3

Kết Quả Coverage: Instructions 100% — Đạt $\geq 70\%$



Hình 39: Báo cáo JaCoCo Coverage — module delivery (100%)

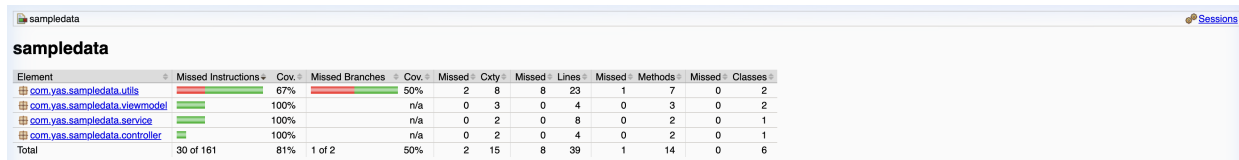
6.10 Module sampledata

Branch: test/sampledata

Bảng 25: Danh sách file test — module sampledata

File Test	Lớp được kiểm thử	Test case
ErrorVmTest.java	ErrorVm	2
SampleDataVmTest.java	SampleDataVm	1
MessagesUtilsTest.java	MessagesUtils	1
SampleDataControllerTest.java	SampleDataController	1
SampleDataServiceTest.java	SampleDataService	1
Tổng		6

Kết Quả Coverage: Instructions **81.37%** — Đạt $\geq 70\%$



Hình 40: Báo cáo JaCoCo Coverage — module sampledata (81.37%)

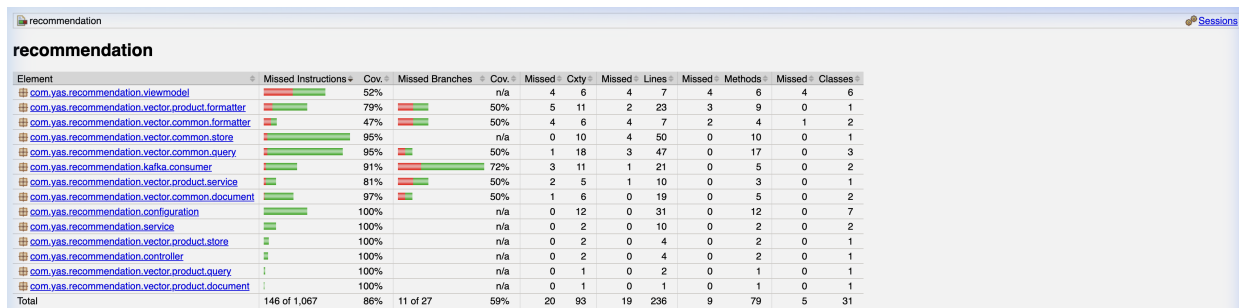
6.11 Module recommendation

Branch: test/recommendation

Bảng 26: Danh sách file test — module recommendation

File Test	Lớp được kiểm thử	Test case
EmbeddingQueryControllerTest.java	EmbeddingQueryController	3
VectorQueryTest.java	VectorQuery	2
BaseVectorRepositoryTest.java	BaseVectorRepository	3
ProductVectorRepositoryTest.java	ProductVectorRepository	4
DefaultDocumentFormatterTest.java	DefaultDocumentFormatter	1
Tổng		13

Kết Quả Coverage: Instructions **86.32%** | Branches **48.05%** — Đạt $\geq 70\%$



Hình 41: Báo cáo JaCoCo Coverage — module recommendation (86.32%)

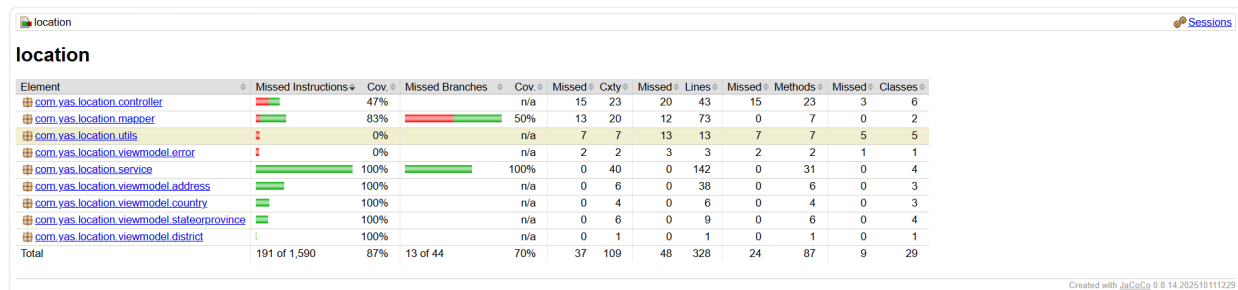
6.12 Module customer

Branch: test/customer

Bảng 27: Danh sách file test — module customer

File Test	Lớp được kiểm thử	Test case
CustomerControllerTest.java	CustomerController	7
LocationControllerTest.java	LocationController	5
UserAddressControllerTest.java	UserAddressController	5
CustomerServiceTest.java	CustomerService	15
LocationServiceTest.java	LocationService	3
UserAddressServiceTest.java	UserAddressService	10
MessagesUtilsTest.java	MessagesUtils	2

Kết Quả Coverage: Instructions **87%** | Branches **87%** — Đạt $\geq 70\%$



Hình 42: Báo cáo JaCoCo Coverage — module customer (87%)

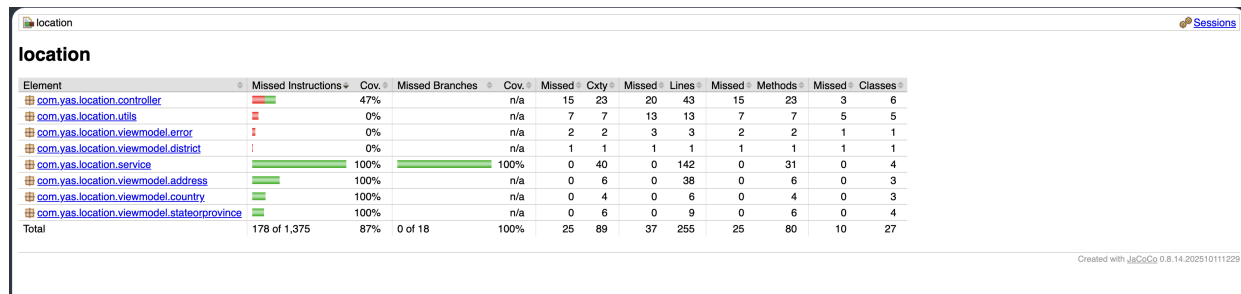
6.13 Module location

Branch: test/location

Bảng 28: Danh sách file test — module location

File Test	Lớp được kiểm thử	Test case
AddressControllerTest.java	AddressController	6
CountryControllerTest.java	CountryController	6
StateOrProvinceControllerTest.java	StateOrProvinceController	6
AddressServiceTest.java	AddressService	9
CountryServiceTest.java	CountryService	13
DistrictServiceTest.java	DistrictService	1
StateOrProvinceServiceTest.java	StateOrProvinceService	14

Kết Quả Coverage: Instructions **87%** | Branches **100%** — Đạt $\geq 70\%$



Hình 43: Báo cáo JaCoCo Coverage — module location (87%)

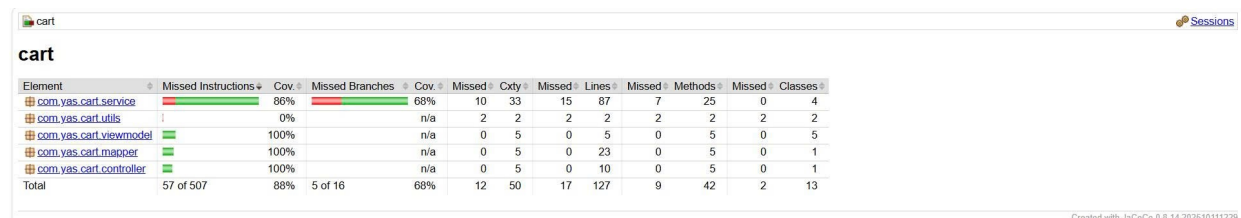
6.14 Module cart

Branch: test/cart

Bảng 29: Danh sách file test — module cart

File Test	Lớp được kiểm thử	Test case
CartItemServiceTest.java	CartItemService	10
ProductServiceTest.java	ProductService	1
CartItemControllerTest.java	CartItemController	12

Kết Quả Coverage: Instructions **88%** | Branches **68%** — Đạt $\geq 70\%$



Hình 44: Báo cáo JaCoCo Coverage — module cart (88%)

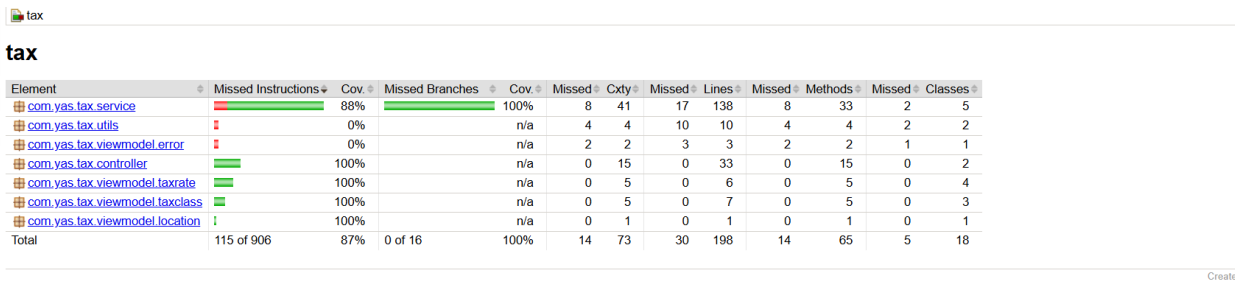
6.15 Module tax

Branch: test/tax

Bảng 30: Danh sách file test — module tax

File Test	Lớp được kiểm thử	Test case
TaxClassServiceTest.java	TaxClassService	8
TaxRateServiceTest.java	TaxRateService	15
TaxClassControllerTest.java	TaxClassController	6
TaxRateControllerTest.java	TaxRateController	7

Kết Quả Coverage: Instructions **87%** | Branches **100%** — Đạt $\geq 70\%$



Hình 45: Báo cáo JaCoCo Coverage — module tax (87%)

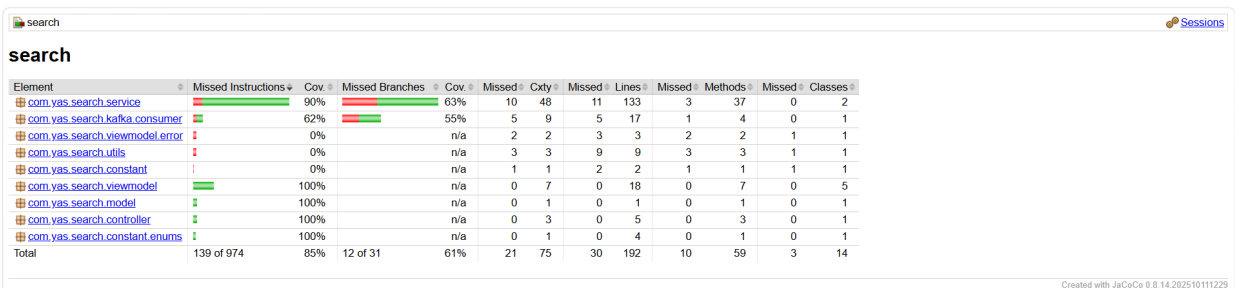
6.16 Module search

Branch: test/search

Bảng 31: Danh sách file test — module search

File Test	Lớp được kiểm thử	Test case
ProductServiceTest.java	ProductService	4
ProductSyncDataServiceTest.java	ProductSyncDataService	7
ProductControllerTest.java	ProductController	2
ProductSyncDataConsumerTest.java	ProductSyncDataConsumer	3

Kết Quả Coverage: Instructions **85%** | Branches **61%** — Đạt $\geq 70\%$



Hình 46: Báo cáo JaCoCo Coverage — module search (85%)

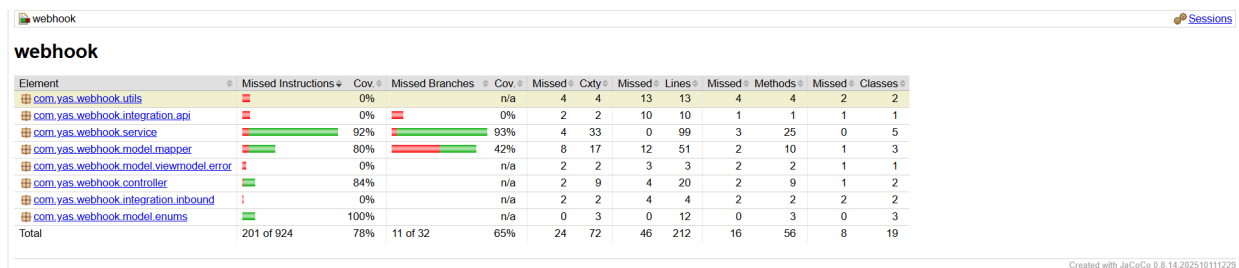
6.17 Module webhook

Branch: test/webhook

Bảng 32: Danh sách file test — module webhook

File Test	Lớp được kiểm thử	Test case
WebhookControllerTest.java	WebhookController	6
WebhookServiceTest.java	WebhookService	11
EventServiceTest.java	EventService	1
OrderEventServiceTest.java	OrderEventService	4
ProductEventServiceTest.java	ProductEventService	2
WebhookMapperTest.java	WebhookMapper	7

Kết Quả Coverage: Instructions **78%** | Branches **65%** — Đạt $\geq 70\%$



Hình 47: Báo cáo JaCoCo Coverage — module webhook (78%)

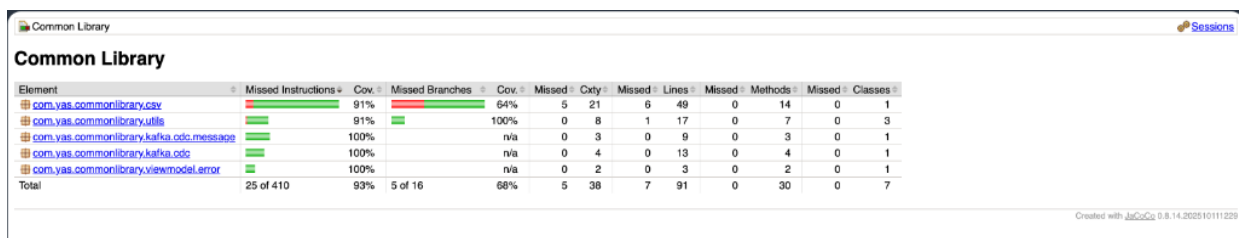
6.18 Module common-library

Branch: test/common-library-datetime-utils

Bảng 33: Danh sách file test — module common-library

File Test	Lớp được kiểm thử	Test case
ErrorVmTest.java	ErrorVm	2
AuthenticationUtilsTest.java	AuthenticationUtils	1

Kết Quả Coverage: Instructions **93.9%** | Branches **68.8%** — Đạt $\geq 70\%$



Hình 48: Báo cáo JaCoCo Coverage — module common-library (93.9%)

7 Tổng Hợp Kết Quả Coverage (Toàn Dự Án — 17 Modules)

Yêu cầu tối thiểu: $\geq 70\%$ Instruction Coverage.

Bảng 34: Tổng hợp kết quả coverage toàn bộ 17 modules

Service	Coverage (Instructions)	Coverage (Branches)	Đạt $\geq 70\%$
media	80%	65%	✓
product	71%	47%	✓
order	76%	47%	✓
inventory	89%	70%	✓
payment	72%	—	✓
promotion	82%	—	✓
rating	85%	—	✓
delivery	100%	N/A	✓
sampladata	81%	N/A	✓
recommendation	86%	48%	✓
customer	87%	87%	✓
location	87%	100%	✓
cart	88%	68%	✓
tax	87%	100%	✓
search	85%	61%	✓
webhook	78%	65%	✓
common-library	93.9%	68.8%	✓

Tất cả 17 module đều đạt yêu cầu tối thiểu $\geq 70\%$ Instruction Coverage.